HOCH
SCHULE
OFFEN
BURG

Fakultät Medien

Bachelorarbeit

Post-Quanten-Kryptographie (PQC) in DNSSEC

Verfasser:	Mark Becher
Matrikelnummer:	188118
Erstprüfer:	Prof. Dr. Erik Zenner
Zweitprüfer:	Prof. Dr. Dirk Westhoff
Fakultät:	Medien

Offenburg, 9. November 2026

Inhaltsverzeichnis

1 Eidesstatliche Erklärung **6**

2 Einleitung **7**

2.1 Einleitung 7

2.2 DNS 9

2.3 Quantencomputer 10

2.3.1 Prinzipien von Quantencomputern 12

3 Hauptteil **17**

3.1 DNS und DNSSEC 17

3.1.1 Upgrade auf PQC 19

3.2 Quantenalgorithmen 20

3.2.1 Logische Gatter und Komponenten 20

3.2.2 Deutsch-Algorithmus 21

3.2.3 Deutsch-Jozsa-Algorithmus 24

3.2.4 Simon's Algorithmus 26

3.2.5 Grover's Algorithmus 27

3.2.6 Shor's Algorithmus 29

3.3 Post-Quanten-Kryptografie Signatur Algorithmen 33

3.3.1 Der Umstieg von derzeitigen Digitalen Signaturen zu PQC 33

3.3.2 NIST Auswahlprozess für Digitale Signaturen 33

3.3.3 weitere PQC-Signatur-Verfahren 35

3.4 Proof-of-Concept: Performance testing der PQC-Algorithmen 37

3.4.1 Benchmarking Auswertung 38

3.5 Merkle Tree Ladder (MTL) Modus 43

4 Fazit und Ausblick **46**

Tabellenverzeichnis

3.1 von DNSSEC Unterstützte Signatur Algorithmen 19
3.2 NIST PQC-Signatur Standarts 20
3.3 Funktionswerte für f(x) 21
3.4 NIST Sicherheitslevel für PQC-Verfahren 33
3.5 PQC-Algorithmen im PoC 38
3.6 Auswertung von keypair im Laufzeitvergleich 39
3.7 Auswertung der Signaturerstellung im Laufzeitvergleich 40
3.8 Auswertung der Verifikation im Laufzeitvergleich 40
3.9 Median der unterschiedlichen Algorithmen 41
3.10 Variation der unterschiedlichen Algorithmen 42

Abbildungsverzeichnis

2.1 output von „host example.com“ 9
2.2 Beispiel der hierarchischen Struktur des DNS-Namensraums 10
2.3 Analoger 10-qubit Quantencomputer von Barcelona's Supercomputing Center (BSC) [38] 11
2.4 Grafische Darstellung eines Qubit als Bloch-Kugel. [17] 14

3.1 RRSIG von example.com 18
3.2 Schaltkreis des Deutsch-Algorithmus [46] 22
3.3 Orakel des deutsch algorithmus [10] 23
3.4 Schaltkreis des Deutsch-Jozsa-Algorithmus [2] 25
3.5 Schaltkreis für Simon's Algorithmus 26
3.6 Funktionsgrafen von 2^n mod 15, nach eigener Darstellung 31
3.7 Schaltkreis für die Periodenfindung 32
3.8 2-dimensionales Gitter mit einem Punkt und dem nächstgelegenen Gitterpunkt [28]. 34
3.9 Binärbaume des Merkle-Baumes für sieben Nachrichten 43
3.10 Ladder für sieben Nachrichten 44
3.11 Binärbaum für acht Nachrichten 44
3.12 Ladder für acht Nachrichten 45
3.13 Gültiger DNS-Domänenbaum einer Domäne/Wurzel mit drei labels/Kindern . . . 45

4.1 Entwicklungsstand Quantencomputer nach BSI [3] 46
4.2 Geschäftlicher Nutzen von Quantencomputing über die Zeit. Eigene Darstellung nach [15]. 47
4.3 Verhältnis der Komplexitätsklassen BPP, NISQ und BQP. Eigene Darstellung. . . 48
4.4 Screenshot, Wireshark captures eines RRSIG-Paketes mit ML-DSA44 Signatur . 49

Algorithmenverzeichnis

1 shor's Algorithmus 30

1 Eidesstatliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Abschlussarbeit selbstständig und ohne unzulässige fremde Hilfe angefertigt habe. Insbesondere versichere ich, dass ich alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen, unveröffentlichten Unterlagen und Gesprächen entnommen wurden, als solche durch Zitate kenntlich gemacht habe. Die vorliegende Abschlussarbeit lag in gleicher oder ähnlicher Fassung noch keiner Prüfungsbehörde vor und wurde bisher nicht veröffentlicht. Ich bin mir bewusst, dass eine falsche Versicherung rechtliche Folgen haben wird. Einer Prüfung der Inhalte und angegebenen Quellen durch eine Plagiatssoftware stimme ich zu.

2 Einleitung

2.1 Einleitung

Durch die Fortschritte in der Entwicklung von Quantencomputern steigt die Notwendigkeit von kryptographischen Verfahren, die auch gegen Angriffe von Quantencomputern resistent sind.

Verfahren für digitale Signaturen wie Diffie-Hellman, DSA, oder das ElGamal Signaturverfahren bauen auf dem diskreten Logarithmusproblem von großen Zahlen auf. Formal bedeutet das, wir haben eine zyklische Gruppe $\mathbb{Z}_p^*$ der Ordnung p-1 (wobei p eine Primzahl ist). Wir brauchen ein primitives element $\alpha \in \mathbb{Z}_p^*$ und ein weiteres Element $\beta \in \mathbb{Z}_p^*$. Das diskrete Logarithmusproblem lautet, ist eine ganze Zahl x in $1 \leq x \leq p-1$ zu finden, dass gilt:

$$\alpha^x \equiv \beta \mod p \tag{2.1}$$

Um x zu finden man berechnen:

$$x = \log_\alpha \beta \mod p \tag{2.2}$$

Für einen herkömmlichen Computer ist das ein schwieriges Problem zu berechnen. Das exponentieren ist einfach zu berechnen, den logarithmus zu berechnen, bei großen Zahlen, nicht. So etwas wird als Einwegfunktion bezeichnet [54].

In der Praxis wird häufig die generalisierung des diskreten Logarithmusproblems verwendet, elliptische Kurven Kryptografie. Elliptische Kurven Kryptografie hat den Vorteil dass sie ein vergleichbares Sicherheitsniveau wie diskrete Logarithmus Varianten aufweist, mit kürzeren Schlüssellängen. DNSSEC unterstützt auch Edwards Curve digital signature algorithm. Es ist ein elliptischer Kurven Algorithmus der dazu entwickelt wurde extra schnell zu sein. DNSSEC unterstüzt zudem auch RSA, obwohl dieser mit einem modulo von 4096 Bit die größten Schlüssellängen der klassischen Verfahren hat.

DNS ist eines der wichtigsten Protokolle, die die Kommunikation im Internet ermöglichen. Es wurde allerdings nie mit dem Gedanken von Sicherheit entworfen. DNS bietet als einfaches

Protokoll keine Sicherheitsfeatures, die DNS Pakete werden im Klartext übertragen und bieten keine Authentizität. Dadurch entstehen für klassisches DNS viele Angriffsvektoren, deshalb wurde DNSSEC entwickelt, mit Hilfe digitaler Signaturen, aus der asymmetrischen Kryptografie soll Integrität und Authentizität gewährleistet werden. Vertraulichkeit ist damit nicht gegeben. Wenn Vertraulichkeit notwendig ist, kann das durch DNS-over-HTTPS oder DNS-over-TLS gewährleistet werden. BIND9 unterstützt auch Kerberos zum austausch von DNS Daten[1]. Auf das Thema Vertraulichkeit in DNS, wird in dieser Bachelorarbeit nicht weiter eingegangen.

In den 90er Jahren veröffentlichte Peter Shor eine Arbeit mit dem Title „Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer“, darin zeigte er, dass es theoretisch möglich wäre, mit einem Quantencomputer, effizient, diskrete Logarithmen zu berechnen und auch Primfaktorzerlegung, das Problem auf dem RSA basiert[64]. 2024 wurde von dem Theoretischen Informatiker Oded Regev ein Algorithmus vorgestellt, der sogar nochmals effizienter ist als Shor's Algorithmus[57]. Durch die Möglichkeit durch Quantenalgorithmen effizient die mathematischen Probleme zu berechnen, auf denen einige Verfahren der asymmetrischen Kryptografie basieren, entstand die Notwendigkeit für Kryptografie die auch gegen Angriffe von Qunantencomputern resistent ist. Die Verfahren der Post-Quanten-Kryptografie befinden sich teilweise noch im NIST-Auswahlverfahren. Was DNSSEC betrifft, ist der wesentliche Unterschied von klassischen Verfahren und Post-Quanten-Kryptografie (PQC) dass PQC deutlich größere Schlüsselllängen haben. Die Schlüssellängen sind für das DNS Protokoll besonders problematisch, im Vergleich zu TLS beispielsweise, da DNS standardmäßig über UDP läuft und eine maximale größe hat, die bei PQC schnell überschritten werden kann [49, 50].
Wie DNSSEC mit PQC Algorithmen umgehen kann, wie das NIST dieses Problem betrachtet und wie es praktisch umgesetzt werden kann, ist die zentrale Fragestellung dieser Bachelorarbeit.

Im ersten Kapitel, Kapitel-2.2 wird das DNS Prokoll kurz vorgestellt und dessen Funktionalität, welche für die Arbeit relevant ist. Anschließend wird in Kapitel-2.3 grundlegend darauf eingegangen wie Quantencomputer funktionieren und was sie von klassischen Computern unterscheidet. Im Hauptteil wird in Kapitel-3.1 die DNSSEC erweiterung vorgestellt und um welche, für diese Arbeit relevanten, funktionalitäten sie DNS erweitert. Das Kapitel, Kapitel-3.3 stellt einige der PQC Algorithmen vor, die zurzeit im NIST-Auswahlverfahren stehen. Die vorgestellten Algorithmen sind entweder interessant weil sie für DNSSEC geeignet sind, oder weil sie ein gutes Beispiel für eine Klasse von Algorithmen sind, wie z.B. Gitterbasiert wie CRYSTALS-Dilithium, FALCON oder Multivariat wie Rainbow, UOV. Im laufe dieser Arbeit wurde ein Proof-of-Concept entwickelt, welches hier in Kapitel-3.4 vorgestellt wird. Das Proof-of-Concept ist ein umgebauter

DNS Server, der die PQC-Algorithmen implementiert hat und PQC-DNSKEY nutzt, dadurch lassen sich Paketlimitierungen und TCP-Fallback veranschaulicht zeigen. Ebenso ist durch die Auswertung eines benchmarkings zu sehen, wie sich die verschiedenen Algorithmen in deren Performance unterscheiden. Es wird in folgenden Kapitel, Kapitel-3.5 der Merkle-Tree-Ladder mode vorgestellt, der von der NIST für die Implementierung für eine PQC Lösung für DNSSEC empfohlen wird. Abschließend werden in Kapitel-4 in einem Fazit die Erkenntnisse aus der Arbeit zusammengefasst, eine Sicht der aktuellen Lage von Quantencomputern gegeben, welche Chancen sich durch Quantenkryptografie bieten und zuletzt eine abschließende Zusammenfassung.

2.2 DNS

Im Internet kennen sich Computer über IP-Adressen. Da es für Menschen nicht realistisch ist, sich für jeden Computer der angesprochen werden soll, sich die IP-Adresse des Computers zu merken, gibt es das Domain Name System (DNS). DNS ist ein ein System, das dazu da ist, IP-Adressen in Namen umzuwandeln die für Menschbar lesbar und umgänglich sind. Ebenso kann DNS auch Namen in IP-Adressen umwandeln, was als reverse-DNS bezeichnet wird. An einem praktischen Beispiel lässt sich das einfach zeigen. Z.B. steht die IP-Adresse von example.com mit **host example.com** abrufen, in Abbildung-2.1 ist ein DNS-Eintrag den man mit „host“ (unter linux) abrufen kann, alternativ lässt sich dies auch mit z.B. „dig“ auflösen.

```
example.com has address 104.20.23.154
example.com has address 172.66.147.243
example.com has IPv6 address 2606:4700:10::6814:179a
example.com has IPv6 address 2606:4700:10::ac42:93f3
example.com mail is handled by 0 .
example.com has HTTP service bindings 1 . alpn="h3,h2" ipv4hint=104.20.23.154,17
```

Abbildung 2.1: output von „host example.com“

DNS ist ein wichtiger Dienst um kommunikation im Internet zu ermöglichen, meistens ist es sogar einer der ersten Dienste die überhaupt die Kommunikation im Internet unter Computern ermöglicht.

DNS wurde entworfen möglichst leichtgewichtig zu sein, mit kleinen Paketen und wenig overhead (durch UDP), und dadurch auch geringeren Latenzen. Das DNS Protokoll läuft standartmäßig auf UDP mit Port 53. TPC-fallback ist möglich, wenn die Nachrichten wegen der Größe fehlschlagen sollten, es ist aber auch möglich direkt DNS Pakete über TCP zu senden, mit gesetztem Bit[49, 11]. Die UPD Nachrichtengröße ist auf 512 Byte beschränkt, für DNS Nach-

richten reicht das eigentlich aus[49, 50]. Die Domänen werden wie in 2.2 zu sehen ist, in einer Baumstruktur angeordnet.

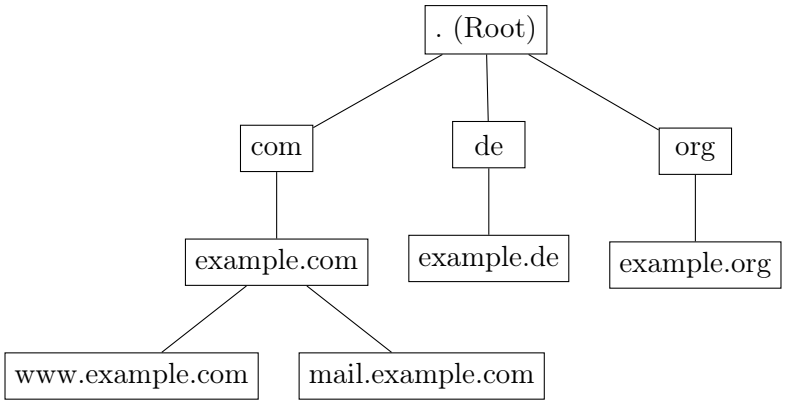


Abbildung 2.2: Beispiel der hierarchischen Struktur des DNS-Namensraums

Der DNS Baum teilt sich anhand dessen Domänen auf. Die oberste Domäne ist die sogenannte root-Domain und wird mit einem „.“ gekennzeichnet. Unter der root-Domain liegen die Top-Level-Domains (TLD). In Abbildung-2.2 sind com, de und org die TLDs, es sind immer die Domänen, die unmittelbar unter der root-Domain liegen. Die Knoten darunter, die keine Blätter sind, sind weitere Domänen, wenn sie weitere Kinder haben, wie in Abbildung-2.2, com, de, org und auch example.com, dann werden diese als Subdomänen bezeichnet. Die Blätter werden als „labels“ bezeichnet. Wie die DNS-Server verteilt sind ist unterschiedlich, ihre zuordnung ist rein logisch, Domänen-Server sind verteilte Datenbanksysteme und die einzelnen Computer können unterschiedlich auf der ganzen Welt verteilt sein[29].

In Abbildung-2.2 ist eine mögliche Struktur einer hierarchischen Struktur eines DNS-Namensraumes zu sehen. Die Namensauflösung geschieht von rechts nach links. Die jeweilige Zone kann mehrere labels haben oder wiederrum subdomänen. Im Beispiel von Abbildung-2.2 haben org und de, jeweils ein label namens example. Das der Name example in beiden Domänen vorkommt, führt zu keinem Konflikt. Trotz dass der Name der gleiche ist, kann unterschieden werden, weil dieser nicht in der selben Domäne ist. Im Fall von der com Domäne ist example wieder eine subdomäne, die die labels „www“ und „mail“ besitzt.

2.3 Quantencomputer

Ein häufiges Missverständnis von Quantencomputern ist, dass sie als eine Art Supercomputer gesehen werden, die in der Lage sind alles was ein herkömmlicher Computer können zu machen, nur viel schneller. Auch häufig hört man die Erklärung, dass ein Quantencomputer im Gegensatz zu einem traditionellen Computer, nicht sequenziell rechnet, sondern alle berechnungen parallel

auszuführen kann. Das ist schon eher korrekt, aber auch noch ungenau.

Quantencomputer sind keine neuen Computer, sie können theoretisch manche Ressourcenlastingen Berechnungen effizienter ausführen, als es ein traditioneller Computer könnte. Sie sind fundamental neue Rechenmodelle. Ein gewöhnlicher Computer funktioniert mit Bits und Bitmanipulationen durch logische Gatter, wie AND, OR, NOT, NAND, NOR, XOR und XNOR. Ein Quantencomputer basiert auf Qubits und benutzt Quantengatter wie das Hadamard-Gatter oder CNOT zur manipulation der Qubits. Zudem geht ein Quantencomputer anderst vor, bei der Berechnung von Problemen und wird mit sogenannten Quantenalgorithmen programmiert. Quantencomputer sind nicht so weitflächig einsetzbar wie klassische Computer, die bessere Bezeichnung wäre eher Quantenprozessor oder Quantenchip. Allerdings hat sich der Begriff Quantencomputer durchgesetzt.

Zur Zeit dieser Arbeit, stecken Quantencomputer noch in ihren Kinderschuhen. Es gibt bereits Fortschritte (IBM quantum error-correction code in 2024 [5, 65]) aber noch ist ein allgemeiner praktischer Einsatz von Quantencomputern noch nicht in Sicht. Das Jahr in dem die ersten funktionsfähigen Quantencomputer geben wird, wird als als Y2Q bezeichnet, der genaue Tag als Q-Day.[62]

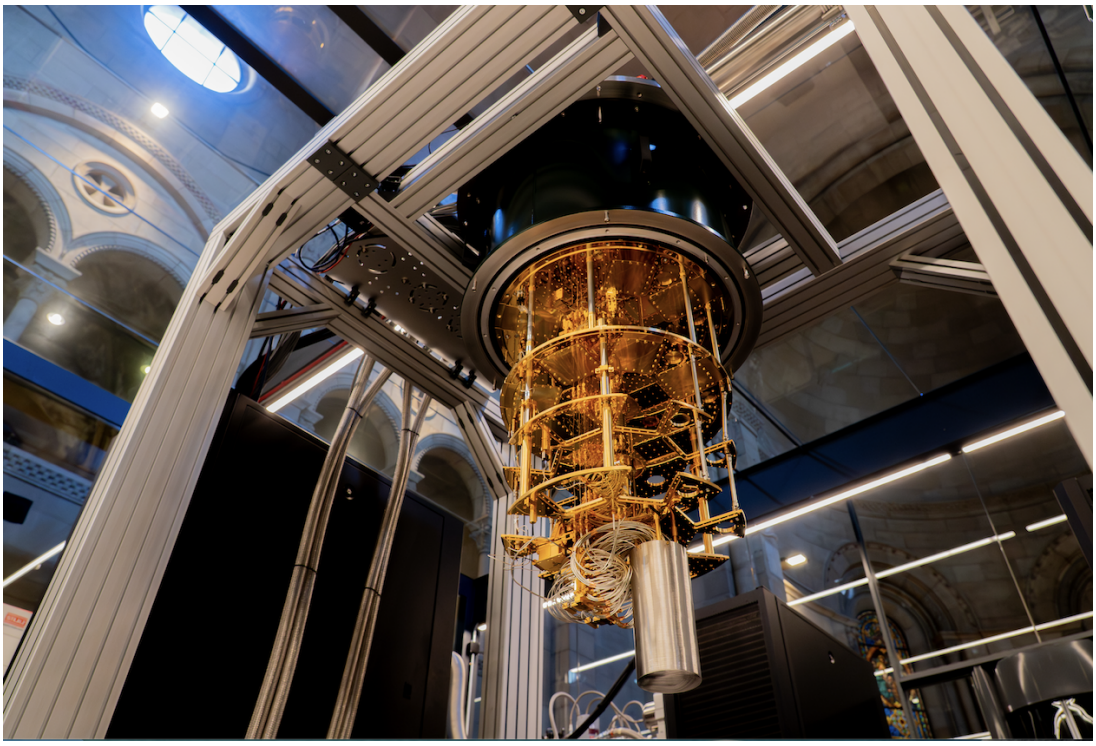


Abbildung 2.3: Analoger 10-qubit Quantencomputer von Barcelona's Supercomputing Center (BSC) [38]

In Abbildung-2.3 ist ein analoger 10-qubit Quantencomputer für 8.5mio € zu sehen, der durch die EU und Spanien finanziert wurde.

2.3.1 Prinzipien von Quantencomputern

Ein Quantencomputer macht sich die Gesetze der Quantenmechanik zu nutze, genauer gesagt Superposition, Quantenverschränkung, Interferenz und Dekohärenz. Um einen Quantencomputer zu betreiben, muss der Prozessor in Themperaturen nahe dem absoluten Nullpunkt betrieben werden. Der Prozessor, welcher das Herz des Computers ist, ist nur eine kleiner Chip, der Rest der Maschine ist vor allem dazu da, die richtigen Bedingungen zu gewährleisten. Andere Technologien als Photonen, wie der Ansatz von Qubits, basierend auf Ionenfallen, biergt die Hoffnung dass Qubits auch bei Zimmertemperaturen betrieben werden können. [56]

Qubits

Ein klassischer Computer arbeitet mit Bits. Bits können einen festen Wert von null oder eins annehmen. Ein Qubit andererseits, kann mit einer bestimmten Wahrscheinlichkeit, Zustände von 0 oder 1 annehmen. Ein Qubit wird als linearkombination (siehe Gleichung-2.3) dargestellt mit den Amplituden α und β.

$$|\psi\rangle \;=\; \alpha\,|0\rangle \;+\; \beta\,|1\rangle \tag{2.3}$$

Die Wahrscheinlichkeit für eine null, ist hier der $P(0) \;=\; |\alpha|^2$ und die Wahrscheinlichkeit auf eine eins, $P(1) \;=\; |\beta|^2$. Folgend, ein Beispiel um es zu verdeutlichen. Man stellt das Qubit als Vektor ψ in ket Notation da. Nehmen wir an wir haben für die Amplituden $\alpha = \sqrt{\frac{1}{8}}$ und $\beta = \sqrt{\frac{7}{8}}$.

$$|\psi\rangle \;=\; (\sqrt{\frac{1}{8}})\,|0\rangle \;+\; (\sqrt{\frac{7}{8}})\,|1\rangle \tag{2.4}$$

Das bedeutet wir haben die Wahrscheinlichkeiten:

$$P(0) \;=\; |\alpha|^2 \;=\; \sqrt{\frac{1}{8}}^2 \;=\; \frac{1}{8} \tag{2.5}$$

$$P(1) \;=\; |\beta|^2 \;=\; \sqrt{\frac{7}{8}}^2 \;=\; \frac{7}{8} \tag{2.6}$$

Die Wahrscheinlichkeiten ergeben in der Summe $|\alpha|^2 + |\beta|^2 \;=\; \sqrt{\frac{1}{8}}^2 + \sqrt{\frac{7}{8}}^2 \;=\; \frac{1}{8} + \frac{7}{8} \;=\; 1.$

Wenn wir mehr als ein Qubit haben, wie hier z.B. 7, können wir damit $2^7 = 128$ Zustände darstellen. Wenn die Zustände eines Qubits von denen eines anderen beeinfluss werden, spricht

man davon, dass diese Quantenverschränkt sind.[62]

$$\alpha_0 \left|0000000\right\rangle + \alpha_1 \left|0000001\right\rangle + \cdots + \alpha_{127} \left|1111111\right\rangle \tag{2.7}$$

Auch bei einem acht-qubit system wie in Gleichung-2.7 Wahrscheinlichkeiten müssen zusammen wieder eins ergeben.

$$|\alpha_0|^2 + |\alpha_1|^2 + \cdots + |\alpha_{127}|^2 = 1 \tag{2.8}$$

Man kann sich den Vektor als Punkt im 128-dimensionalem Koordinatensystem darstellen. Durch Quantengatter lassen sich die Zustände manipulieren, dadurch ist es möglich Quantenalgorithmen zu entwerfen. Ein Quantengatter muss vollständig umkehrbar sein und die Summe aller quadrierten Amplituden muss wieder eins ergeben [56].

Eine bekannte Art ein Qubit grafisch darzustellen ist als Bloch-Kugel. Ein Qubit wie in Gleichung-2.3 lässt sich auch in dessen Polarkoordinatendarstellung, wie in Gleichung-2.9 darstellen.

$$|\psi\rangle = \cos(\theta/2)e^{i\varphi_0}|0\rangle + \sin(\theta/2)e^{i\varphi_1}|1\rangle \tag{2.9}$$

[48]

Damit lässt sich ein Qubit grafisch darstellen, wie in Abbilbung-2.4. Dessen Amplituden sind dann winkel. Diese Darstellung hilft dabei, sich das abstrakte Konzept von Qubits bildlich vorstellen zu können, dadurch kann man auch ein etwas „intuitiveres“ Gefühl vom Verhalten von Qubits erhalten.

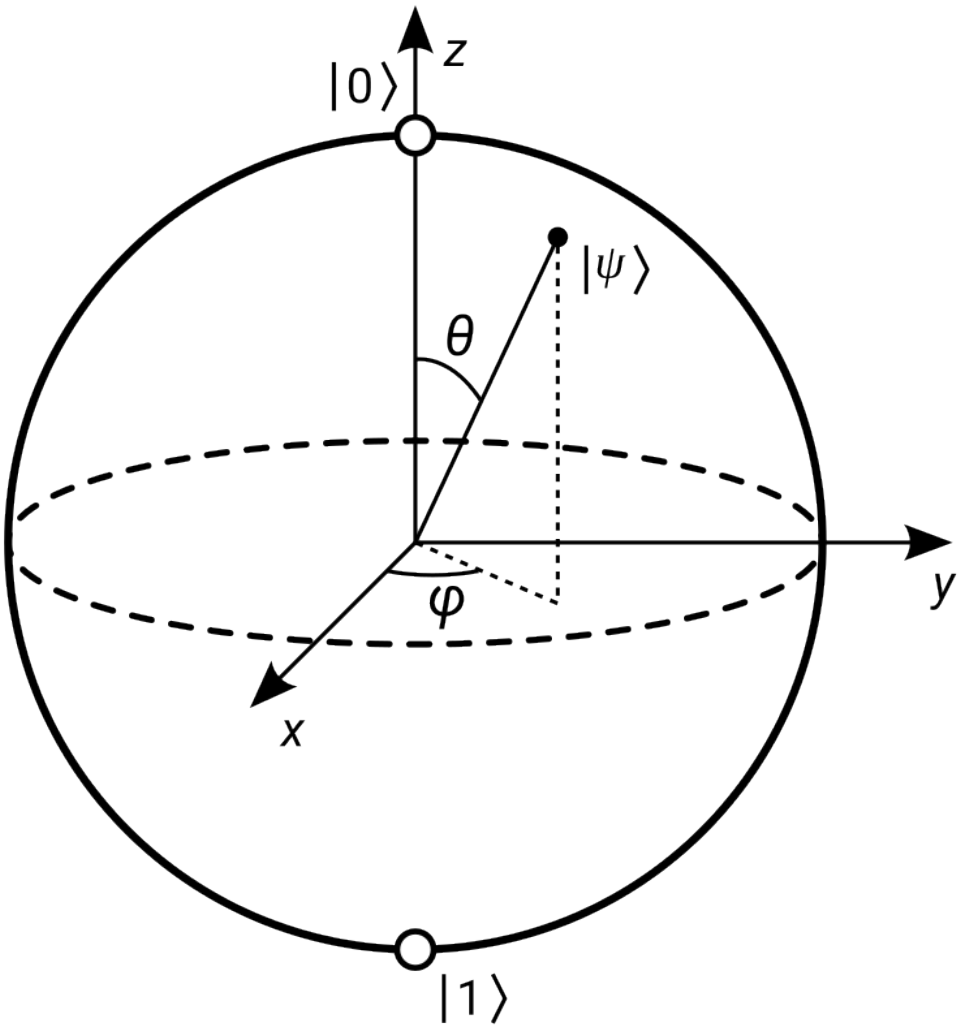


Abbildung 2.4: Grafische Darstellung eines Qubit als Bloch-Kugel. [17]

Überlagerung/Superposition

Wie das anschauliche Beispiel von Schrödingers Katze nahelegt, man kann nicht wissen ob die Katze lebendig oder tot ist, solange man nicht nachschaut, das gleiche Prinzip gilt auch bei Qubits. Qubits können einen Zustand als Teilchen oder als Welle haben, solange man diese nicht observiert, haben sie eine Wahrscheinlichkeit eine Welle zu sein und die Gegenwahrscheinlichkeit für ein Teilchen.

Dieses Phänomen der Physik wird als Überlagerung oder Superposition beschrieben. Es macht es Quantencomputern möglich, mittels einer Anzahl n an Qubits, 2^n Zustände abzubilden.

Ein einzelnes Qubit aus Gleichung-2.10 kann 2^1 Zustände $0, 1$ annehmen. Wenn sich dieses Qubit in der Superposition befindet, befindet es sich mit den Amplituden $\alpha, \beta \in \mathbb{C}$ und den dazugehörigen Wahrscheinlichkeiten $|\alpha|^2$ für eins und mit der Gegenwahrscheinlichkeit $|\beta|^2$ für null.

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle \tag{2.10}$$

Der Zustand eines n-Qubit Quantencomputer lässt sich wie in Gleichung-2.11 darstellen.

$$|\psi\rangle = \sum_{k\,=\,0}^{2^n-1} \alpha\,|c_k\rangle \tag{2.11}$$

mit $c \in \mathbb{C}$ als Amplitude und $k \;=\; [0, 2^n - 1]$ in Binärdarstellung der Qubits. Ein n-Qubit Quantencomputer wie in Gleichung-2.11 erzeugt Wellenfunktionen in einem 2^n-dimensionalem komplexen Hilbertraum[9].

Quantenverschränkung

Ein Qubit alleine ist nicht sonderlich nützlich, damit ein Quantencomputer sein volles Potenzial entfalten kann, ist die Quantenverschränkung unerlässlich. Wenn zwei (oder mehrere) Qubits verschränkt sind, bedeutet dass eine Änderung des einen, auch eine Auswirkung auf das Andere hat.

Ein n-Qubit Quantencomputer wie in Gleichung-2.11 ist nur wirklich nützlich wenn die Amplituden auch voneinander abhängig sind. Wenn sich die Amplitude (damit auch die Wahrscheinlichkeit) eines der n-Qubits ändert, beinflusst dies auch die Anderen.

Interferenz

Im Zusammenspiel mit Superposition und Quantenverschränkung, macht dieses Prinzip Quantencomputer zu extrem Leistungsfähigen Rechenmaschinen. Die Rechenpfade die entstehen, las-

sen sich als komplexe Amplituden ausdrücken, die sich wie Wellen verhalten. Diese Amplituden können sich gegenseitig verstärken (konstruktive Interferenz) oder gegenseitig abschwächen (destruktive Interferenz). Damit können sich die Rechenpfade selbst auf den richtigen Weg lenken, die Wahrscheinlichkeit auf einen richtigen Rechenweg wird verstärkt [51].

Dekohärenz

Qubits sind sehr instabil, sie zerfallen leicht aus einer Superposition in einen feste Zustand. Dekohärenz ist der Prozess in dem ein System aus einem Quantenzustand in einen nicht-Quantenzustand zerfällt. Damit Quantencomputer richtig funktionieren, ist es wünschenswert, die Dekohärenz möglichst zu minimieren. Es kann aber auch beabsichtigt sein, z.B. wenn die Messung nach der Manipulation erfolgt, dadurch verfallen die Qubits in einen festen Zustand, der aussagekräftig für die Berechnung ist [26].

3 Hauptteil

3.1 DNS und DNSSEC

Der standart DNS Service ist anfällig gegen eine vielzahl von Angriffen. Z.B. durch manipulation der DNS Pakete lassen sich andere Zielrechner ansteuern als gedacht. Dadurch dass DNS Einträge aus dem Cache lesen kann, um Latenz zu sparen, sind sogenannte Cache Poisoning Angrifffe möglich, bei dem falsche Adressen in den Cache geladen werden, um künftige Anfragen, falsch weiter zuleiten. Ein DNS Cache poisoning Angriff, läuft folgendermaßen ab:

Angreifer schickt einen gefälschten Eintrag Der Angreifer schickt dem DNS-Server einen vermeintlichen Eintrag für eine Website, dieser Eintrag ist aber nicht die richtige Seite, sondern eine vom Angreifer kontrollierte Seite.

Der DNS-Server nimmt diese an Der DNS-Server hat keinen Beweis für die gütligkeit des Eintrags und nimmt diesen einfach an.

Client fragt nach dem Eintrag Der Client frag nach der Adresse für den gefälschten Eintrag. Der Server antwortet mit dem Eintrag aus seinem Cache.

Erfolgreicher Angriff Das Opfer wurde nun auf die Seite des Angreifers gelockt[25].

Um die Authentizität und Integrität der DNS Einträge zu gewährleisten, gibt es die DNS Security Extensions (DNSSEC). Es muss berücksichtigt werden, dass dadurch aber keine Vertraulichkeit der Daten gewährleistet wird und Angriffe wie DNS-Rebinding Angriffe und D(DoS) Angriffe weiterhin möglich sind.

Durch die DNSSEC Erweiterung kommt folgendes hinzu: Zwei öffentliche Schlüssel:

- Key Signing Keys (KSK)
  signieren die DNS-Einträge innerhalb einer Zone
- Zone Signing Keys (ZSK)
  signieren die KSK

Zudem auch neue Ressource Record Typen:

- Resource Record Signature (RRSIG)
- DNS Public Key (DNSKEY)
- Delegation Signer (DS)
- Next Secure (NSEC)

Resource Record Signature (RRSIG)

Dieser Resource Record (RR) enthält die Signaturen über ein bestimmtes RR Set. Die Signatur, die darin gespeichert ist, ist für einen bestimmten Namen und Typ gültig.

```
; <<>> DiG 9.20.26 <<>> example.com RRSIG
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 27172
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 5

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;example.com.                   IN      RRSIG

;; ANSWER SECTION:
example.com.            2799    IN      RRSIG   DNSKEY 13 2 3600 20261008174644 2
example.com.            86      IN      RRSIG   AAAA 13 2 300 20260903185217 2026
example.com.            83      IN      RRSIG   A 13 2 300 20260903185214 2026090
example.com.            203     IN      RRSIG   HTTPS 13 2 300 20260903185414 202

;; ADDITIONAL SECTION:
example.com.            83      IN      A       172.66.147.243
example.com.            83      IN      A       104.20.23.154
example.com.            86      IN      AAAA    2606:4700:10::ac42:93f3
example.com.            86      IN      AAAA    2606:4700:10::6814:179a

;; Query time: 22 msec
;; SERVER: 192.168.178.1#53(192.168.178.1) (UDP)
;; WHEN: Wed Sep 02 19:55:51 CEST 2026
;; MSG SIZE  rcvd: 556
```

Abbildung 3.1: RRSIG von example.com

In Abbildung-3.1 ist der Output von **dig example.com RRSIG** zu sehen. Das ist ein RRSIG Eintrag der Domäne example.com.

DNS Public Key (DNSKEY)

Der öffentliche Schlüssel für die Zone. Mit diesem RR kann der Resolver die digitalen Signaturen der Zone prüfen. Dieser Eintrag speichert einen Schlüssel und keine Signatur, anderst als RRSIG. Für untergeordnete Zonen wird dieser Eintrag als child DNSKEY (CDNSKEY) bezeichnet.

Delegation Signer (DS)

Verknüpft eine delegierte Kindzone mit der übergeordneten Zone. Es ist ein RR der zum Schützen Delegierung verwendet wird. Es gibt diesen RR auch für untergeordnete Delegierungssignaturgeber, sog. child delegation signer (CDS).

Next Secure (NSEC)

NSEC ist ein Beweis der nicht-existenz einer Zone oder eines Domänennamens. Er verhindert einen Angriff durch Zonenaufzählungen. Die neuste Version des NSEC ist durch RFC 9824 [45] beschrieben [22, 24, 23, 25].

Signatur Algorithmen für DNSSEC

Durch diese Erweiterungen von DNSSEC ist eine vergrößerung der UDP Pakete (Extension Mechanisms for DNS (EDNS0)) notwendig.[71] von DNNSEC werden die Algorithmen aus Abbildung-3.1 derzeit unterstützt.[58]

Signatur Algorithmus	DNSSEC Code	Länge öffentlicher Schlüssel (in Bits)	Signatur Länge (in Bits)
RSA mit SHA-256	8	≥ 2048	$\approx 2050 - 4100$
ECDSA P-256 mit SHA-256	13	256	512
ECDSA P-384 mit SHA-384	14	384	768
Ed225516	15	256	512
Ed448	16	456	912

Tabelle 3.1: von DNSSEC Unterstützte Signatur Algorithmen

Wie aus Abbildung-3.1 anhand des DNS Code 13 entnehmen kann wird, in diesem Paket, ECDSA P-256 mit SHA-256 als Signatur Algorithmus verwendet.

3.1.1 Upgrade auf PQC

Die klassischen Signatur Algorithmen haben bereits große Schlüssel- und Signaturlängen und DNSSEC benötigt eine Erweiterung der UDP Pakete.[71] Die Post-Quantum Signatur Algorithmen gehen teilweise bereits in den KB Bereich. Zum Vergleich in Abbildung-3.2, die aus Runde 4 standartisierten NIST-Standarts für Digitale Signaturen und deren Varianten.

PQC- Algorithmus	Typ	Öffentlicher Schlüssel Länge	Signatur Länge
CRISTALS-DILITHIUM2	Gitterbasiert	1312 Bytes	2420 Bytes
CRISTALS-DILITHIUM3	Gitterbasiert	1952 Bytes	3293 Bytes
CRISTALS-DILITHIUM5	Gitterbasiert	2592 Bytes	4595 Bytes
FALCON-512	Gitterbasiert NTRU	897 Bytes	666 Bytes
FALCON-1024	Gitterbasiert NTRU	1793 Bytes	1280 Bytes
SPHINCS$^+$-128s	Hashbasiert	32 Bytes	8 080 Bytes
SPHINCS$^+$-128f	Hashbasiert	32 Bytes	16 976 Bytes
SPHINCS$^+$-192s	Hashbasiert	48 Bytes	17 064 Bytes
SPHINCS$^+$-192f	Hashbasiert	48 Bytes	35 664 Bytes
SPHINCS$^+$-256s	Hashbasiert	64 Bytes	29 792 Bytes
SPHINCS$^+$-256f	Hashbasiert	64 Bytes	49 216 Bytes

Tabelle 3.2: NIST PQC-Signatur Standarts

Wie in Abbildung-3.2 zu sehen ist, sprengen bereits die kleinsten Signatur- und Schlüssellängen den Rahmen von einfachen UDP Paketen. Jede der SPHINCS$^+$-Varianten kann mit mit den Hashfunktionen, SHAKE oder SHA betrieben werden [55, 8, 63, 36, 31].

3.2 Quantenalgorithmen

Um einen Quantenalgorithmus zu entwickeln, sind fundamental neue logische Operatoren notwendig. Damit ein Algorithmus effizient ist, müssen die Prinzipen, Quantenverschränkung, interferenz und Dekohärenz, effektiv eingesetzt werden.

3.2.1 Logische Gatter und Komponenten

Quantengatter können durch Matrizen als das mathematische Äquivalent dargestellt werden. Eine wichtige Maßnahme dazu ist, dass die Gatter eine Unitäre Matrix sind und das alle Amplituden a, $\sum_i |a_i|^2 = 1$. Ein wichtiges Gatter ist das Hadamard-Gatter, dieses wird an den Qubits $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ und $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ verdeutlicht:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{3.1}$$

Mathematische Definition aus Gleichung-3.1, mit ihr kann eine Qubit in eine Superposition gebracht und auch wieder in eine feste Position gebracht werden. Angewandt auf Qubits, bringen

sie folgende Resultate:

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \tag{3.2}$$

$$H|1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle \tag{3.3}$$

Die Zustände haben zwar die gleiche Wahrscheinlichkeit aus 0 oder 1, allerdings unterscheidet sich ihre phase, welcher entscheident ist für Interferenz. Wenn die Hadamard-Matrix erneut auf $|-\rangle$ oder $|+\rangle$ angewandt wird, ergibt sich wieder der ursprüngliche Zustand 0 oder 1. Wichtig für einen Quantenalgorithmus ist zudem das Orakel / die Blackbox. Quantenalgorithmen wie der Deutsch- oder Deutsch-Jozsa-Algorithmus funktionieren nach dem query model. Es können Anfragen an das Orakel gestellt werden und für jeweils dafür Antworten erhalten, wie häufig Anfragen gestellt werden, bestimmt die effizienz des Algorithmus. Das Orakel wird als unitary query-gate U_f definiert, für das kann eine Anfrage mit x gestellt werden und eine Antwort $f(x)$ erhalten werden. Ein Quantencomputer kann mehrere Eingabe auf einmal an das Orakel stellen, damit wird parallelität und Interferenz deutlich.

3.2.2 Deutsch-Algorithmus

Der Deutsch Algorithmus ist einer der ersten Quantenalgorithmen, der gezeigt hat, dass ein Quantencomputer theoretisch einen Vorteil gegenüber einem klassischen Computern haben kann. Dieser Algorithmus hat keinen praktischen nutzen, ist aber ein gutes Beispiel wie so ein Algorithmus funktionieren kann. Das Problem ist, wir haben eine Funktion f, die entweder konstant oder balanciert ist.

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	1	1
1	0	1	0	1

Tabelle 3.3: Funktionswerte für f(x)

Die Funktion aus Abbilung-3.3 ist balanciert wenn sie wie f_2 und f_3 gleich häufig 0 wie 1 als Ergebnis liefert, im Fall von f_1 und f_4 ist sie konstant. Der Deutsch-Algorithmus funktioniert nur genau für dieses Problem, in dem wir die Funktion $f\colon \Sigma \to \Sigma$ mit $\Sigma \in \{0, 1\}$ haben.

Deutsch-Problem

Input: Eine Funktion $f\colon \Sigma \to \Sigma$ als Orakel.

Promise: f ist konstant oder balanciert.

Output: „konstant“ oder „balanciert“.

der Deutsch Algorithmus kann folgendermaßen dargestellt werden:

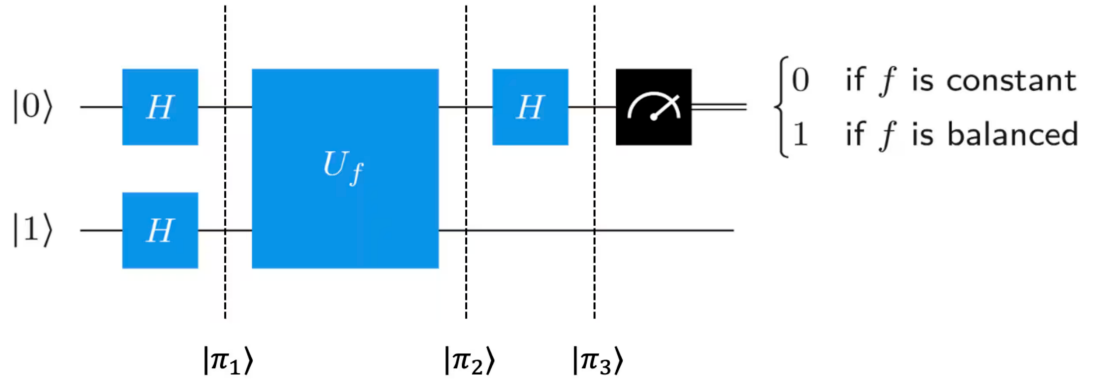


Abbildung 3.2: Schaltkreis des Deutsch-Algorithmus [46]

Wie in Abbildung-3.2 zu sehen ist, werden zuerst zwei Qubits initialisiert mit den Werten 0 und 1. in einem n-qubit system, werden die Qubits über das tensor produkt verbunden. Die Qubits werden absichtlich mit $|0\rangle$ und $|1\rangle$ initialisiert, in das erste Qubit $|0\rangle$ wird der Wert für x rein geschrieben, das andere Qubit wird als hilfsregister verwendet, es wird mit dem Ergebnis der Funktion beschrieben. Wenn wir bei der initialisierung der Qubits, den Zustand $|\pi_1\rangle$ haben, dann wäre dieser:

$$|\pi_0\rangle = |1\rangle|0\rangle \tag{3.4}$$

$$|\pi_0\rangle = |10\rangle \tag{3.5}$$

Als erster Schritt, wird auf beide Qubits jeweils ein Hadamard-Gatter angewandt, im in Abbildung-3.2 beschriebenen Zustand $|\pi_1\rangle$ bilden die Qubits den Zustand:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \tag{3.6}$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle \tag{3.7}$$

$$|\pi_1\rangle = |-\rangle \otimes |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)|1\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle \tag{3.8}$$

Durch ausmultiplizieren erhalten wir:

$$|\pi_1\rangle = \frac{1}{2}(|0\rangle - |1\rangle)|0\rangle + \frac{1}{2}(|0\rangle - |1\rangle)|1\rangle \tag{3.9}$$

Nachdem sich die Qubits in superposition befinden, wird das Orakel befragt. Das ist der Vorteil eines Quantencomputers, dieser kann mit nur einer Abfrage des zwei-Qubit Systems eine Antwort

enthalten, ein klassischer Computer müsste zwei seperate Anfragen für die Fälle 0 oder 1 machen. Das Orakel U_f ist ebenfalls eine unitäre Matrix und wird durch $U_f|x\rangle|y\rangle = |x\rangle|y \otimes f(x)\rangle$ definiert. Durch das Orakel wird das 0-Qubit nicht verändert, das 1-Qubit wird abhängig vom 0-Qubit verändert, wie in 3.3 zu sehen ist.

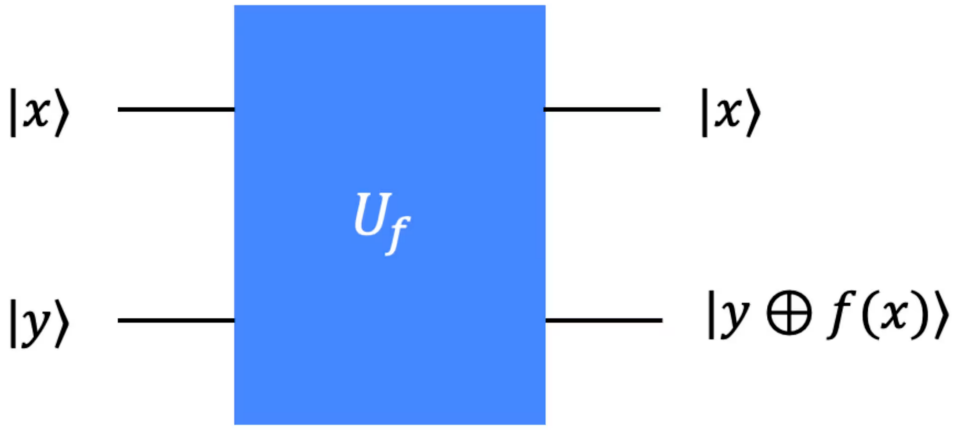


Abbildung 3.3: Orakel des deutsch algorithmus [10]

Das $|0\rangle$ bleibt im $|0\rangle$ Zustand. Das $|1\rangle$ Qubit wird in den Zustand $|1 \oplus f(0)\rangle$ versetzt.
Nachdem wir $|\pi_1\rangle$ darauf ausgeführt haben, erhalten wir den Zustand:

$$|\pi_2\rangle = \frac{1}{2}(|0 \oplus f(0)\rangle - |1 \oplus f(0)\rangle)|0\rangle + \frac{1}{2}(|0 \oplus f(1)\rangle - |1 \oplus f(1)\rangle)|1\rangle \tag{3.10}$$

.

Wenn wir betracht ziehen, dass man auch noch folgendes vereinfachen kann:

$$|0 \oplus a\rangle - |1 \oplus a\rangle = (-1)^a(|0\rangle - |1\rangle) \tag{3.11}$$

Dann lässt sich $|\pi_2\rangle$ auch so schreiben:

$$|\pi_2\rangle = \frac{1}{2}(-1)^{f(0)}(|0\rangle - |1\rangle)|0\rangle + \frac{1}{2}(-1)^{f(1)}(|0\rangle - |1\rangle)|1\rangle \tag{3.12}$$

Interessanterweise hat dabei das Hilfs-Qubit seinen Zustand $|-\rangle$ behalten. Das Eingabe-Qubit erhält abhängig von $f(x)$ eine Phase von $(-1)^{f(x)}$, dadurch wird die Phase auf das Eingabe-Qubit „zurückübertragen“, das wird als **Phase Kickback** bezeichnet und ist wichtig für die Interferenz der verschiedenen Rechenpfade.

Vereinfacht lässt sich dann $|\pi_2\rangle$ noch wie in Gleichung-3.13 darstellen.

$$|\pi_2\rangle = |-\rangle \left(\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \right) \tag{3.13}$$

Wenn der Faktor $(-1)^{f(0)}$ aus der Summe von Gleichung-3.13 gezogen wird, erhalten wir:

$$|\pi_2\rangle = (-1)^{f(0)}|-\rangle \left(\frac{|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle}{\sqrt{2}} \right) = \begin{cases} (-1)^{f(0)}|-\rangle|+\rangle & \text{falls } f(0) \oplus f(1) = 0, \\ (-1)^{f(0)}|-\rangle|-\rangle & \text{falls } f(0) \oplus f(1) = 1. \end{cases} \tag{3.14}$$

Nach der Anwendung des letzten Hadamard-Gatters auf das erste Qubit, erhalten wir den Zustand $|\pi_3\rangle$. Der Zustand $|\pi_3\rangle$ sieht so aus wie in Gleichung-3.15:

$$|\pi_3\rangle = \begin{cases} (-1)^{f(0)}|-\rangle|0\rangle & \text{falls } f(0) \oplus f(1) = 0 \\ (-1)^{f(0)}|-\rangle|1\rangle & \text{falls } f(0) \oplus f(1) = 1 \end{cases} \tag{3.15}$$

Dabei steht das $|-\rangle$ für das Hilfsbit und das andere/rechte Qubit für das Eingabe-Qubit. Nun kann zuletzt die Messung des rechten Qubits erfolgen, dadurch bricht die Superposition zusammen und das Qubit nimmt den Wert 0 oder 1 an. Bei einer konstanten Funktion nimmt das Qubit den Wert 0 an und bei einer balancierten den Wert 1.

Mit einem klassischen Computer müsste man f(0) und f(1) prüfen, um festzustellen ob f konstant oder balanciert ist, mit einem Quantencomputer und dem Deutsch-Algorithmus genügt eine Anfrage. Der Vorteil ist realistisch nicht wahrnehmbar, aber es zeigt das es theoretisch möglich ist einen Quantenvorteil zu schaffen.

Dieser Algorithmus ist bereits kompliziert und abstrakt, er ist aber noch ziemlich simpel im Vergleich zu anderen, die folgend nicht mehr ausführlich erklärt werden.

3.2.3 Deutsch-Jozsa-Algorithmus

Der Unterschied zum Deutsch-Algorithmus ist, dass dieser eine Funktion bearbeiten kann die eine beliebige Anzahl an input Bits nehmen kann und ein Bit als Ausgabe hat. Der Algorithmus bei einer Funktion $f\colon \Sigma^n \to \Sigma$ für alle $n \geq 1$, bestimmen ob diese balanciert oder konstant ist, mit exponentieller Beschleunigung im Gegensatz zu klassischen Algorithmen, einschließlich randomisieren klassischen Algorithmen.

Deutsch-Jozsa Problem

Input: Eine Funktion $f\colon \Sigma^n \to \Sigma$ als Orakel.

Promise: f ist konstant oder balanciert.

Output: „konstant“ 0 oder „balanciert“ 1.

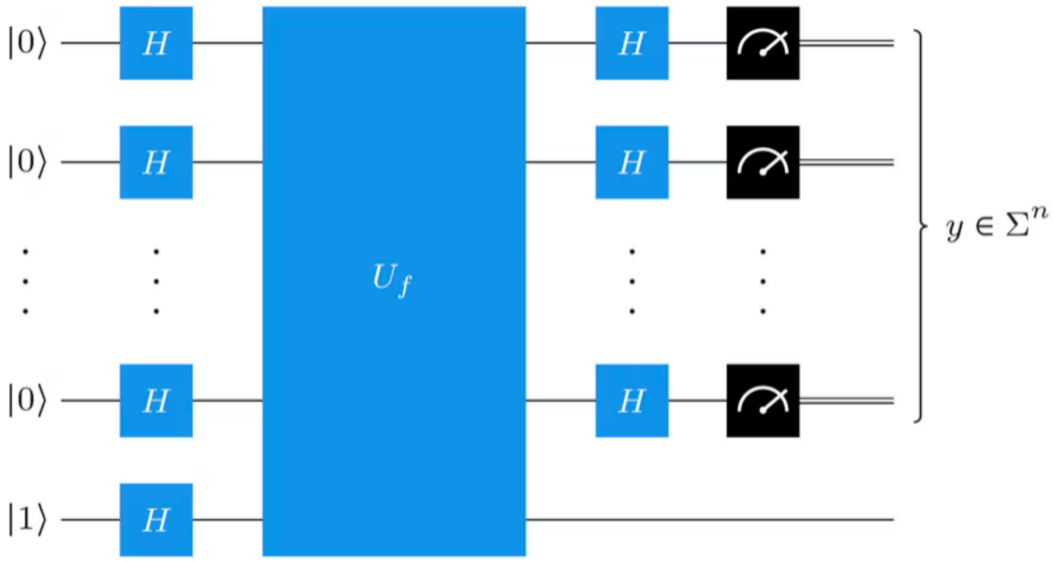


Abbildung 3.4: Schaltkreis des Deutsch-Jozsa-Algorithmus [2]

Interressant zu sehen, ist auch das in 3.4 ein Qubit als $|1\rangle$ initialisiert wird, damit der Phase Kickback später wirken kann.

Wenn alle ouput Qubits 0 sind, dann ist die Funktion f balanciert, falls mindestens ein Qubit den Wert 1 hat, ist sie konstant. Funktionen, die weder balanciert noch konstant sind, werden nicht berücksichtigt. Dieser Quantenalgorithmus benötigt $O(1)$ Operationen um das Problem zu lösen, ein klassischer determisitischer Algorithmus liegt im schlechtesten Fall bei $2^{n-1} + 1$ Anfragen.

Der Deutsch-Jozsa Algorithmus ist eine generalisierung des Deutsch Algorithmus, dieser Algorithmus kann auch auf andere Probleme übertragen werden, wie z.B. auf das Bernstein-Vazirani Problem. Das Bernstein-Vazarani Problem kann wie folgt beschrieben werden:

Bervstein-Verani Problem

Input: $f : \Sigma^n \to \Sigma$

Promise: Es existiert ein binärstring $s = s_{n-1} \dots s_0$ für den gilt, $f(x) = s \times s$ für alle $x \in 0, 1^n$

Output: String s

Dieses Problem lässt sich mit dem Schaltkreis des Deutsch-Jozsa Algorithmus lösen, darauf soll aber nicht weiter eingegangen werden.

3.2.4 Simon's Algorithmus

Dieser Algorithmus ist in der Lage bei einer Funktion f mit $f\colon \Sigma^n \to \Sigma^m$ einen String s zu finden. Mit einem klassischen Computer erfordert das in $O(n^{\frac{n}{2}})$ Orakelabfragen, einen bestimmten String zu finden, das ist im prinzip die Schwierigkeit eine kollision zu finden. Mit Simon's Algorithmus lässt sich das in $O(n)$ schritten lösen.

Simon's Problem

Input: Funktion $f\colon \Sigma^n \to \Sigma^m$

Promise: Es gibt einen String $s \in \Sigma^n$ für den $[f(x) = f(y)] \equiv [(x = y) \vee (x \oplus s = y)]$ für alle $x, y \in \Sigma^n$

Output: String s

In 3.5 kann man sehen, dass sich der Schaltkreis dem vom Deutsch-Jozsa Algorithmus sehr ähnlich sieht.

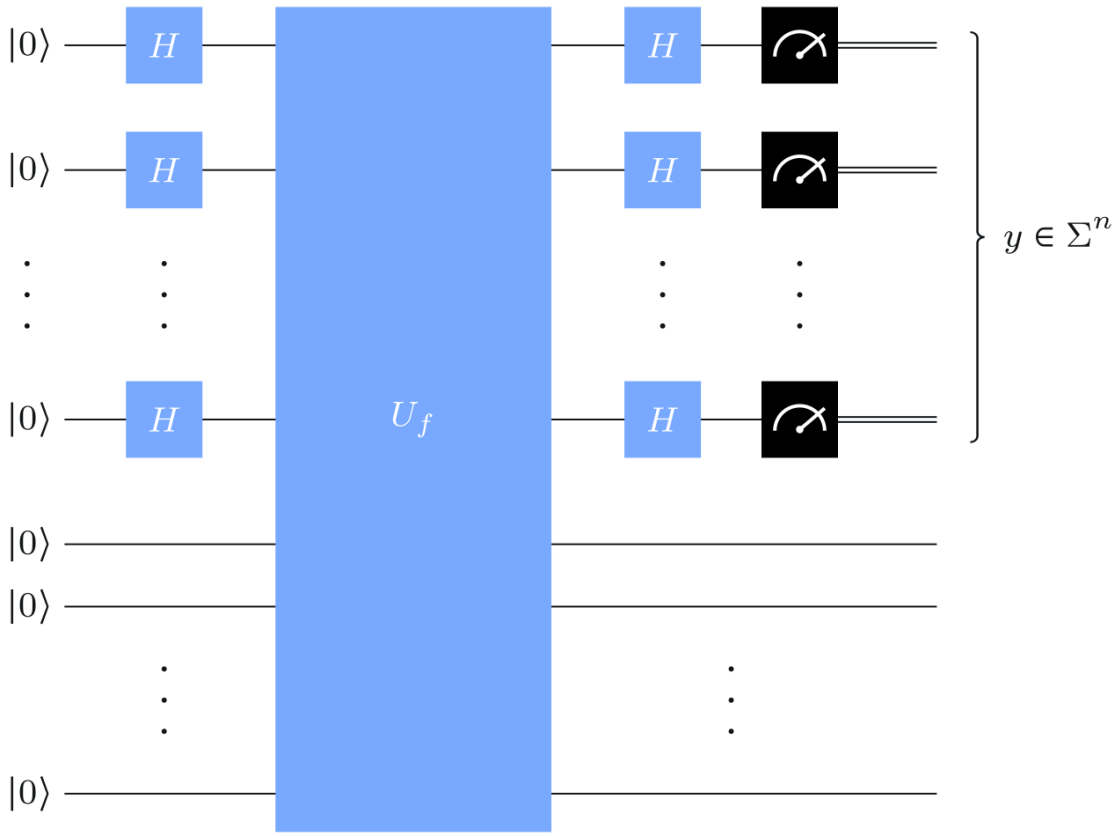


Abbildung 3.5: Schaltkreis für Simon's Algorithmus

Ein entscheidender Unterschied ist, dass dabei alle Qubits als $|0\rangle$ initialisiert werden. Die

Orakel Funktion ist die selbe query model wie bei den vorher genannten:

$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle \tag{3.16}$$

Der gesamte Algorithmus wird aber mehrfach eingesetzt. Es werden $2n$ Qubits initialisiert. Nur n- von 2n-Qubits, werden durch ein Hadamard-Gatter geleitet, alle 2n durch die Orakel Funktion, die ersten n wieder durch ein Hadamard-Gatter und eine Messung der n-Qubits, die durch Hadamard-Gatter geleitet wurden. Dieses Vorgehen wird solange ausgeführt, bis sich mit einer ausreichenden Wahrscheinlichkeit der String bestimmen lassen kann. Dieser Algorithmus ist auch wichtig für heutige NISQ-Quantencomputer, da einige angepasste Versionen, sich mit diesen Computern benutzen lassen. Er war auch die Inspiration für Shor's Algorithmus zum effizienten Faktorisieren von Primfaktoren. Simon's Algorithmus selbst lässt sich in wenigen spezifischen Fällen, lässt sich dieser Algorithmus als Angriff aus Symmetrische Kryptoverfahren einsetzen. Allerding lässt sich Simon's Problem mit probalistischen Algorithmen auch mit klassischen Computern in wenigen Schritten lösen [13, 26, 35].

3.2.5 Grover's Algorithmus

Grover's Algorithmus ist in der Lage einen String in einem unstrukturierten Datensatz zu finden. Klassische Verfahren brauchen für n Strings, durchschnittlich $O(\frac{n}{2})$ Schritte um einen Eintrag zu finden, mit Grover's Algorithmus in das in nur $O(\sqrt{n})$ Schritten möglich. Grover's Algorithmus lässt sich folgendermaßen definieren:

Unstrukturierte Suche

Input: $f : \Sigma^n \to \Sigma$

Output: ein String $x \in \Sigma^n$ für den gilt f(x)=1 oder „keine Lösung", falls kein solcher String existiert

Anstelle ein Hilfs-Qubit zu verwenden um Nutzen des Phase Kickback zu machen, kann dies durch ein *Phase Query Gate* implementiert werden. Das geschieht durch:

$$Z_f : |x\rangle \to (-1)^{f(x)}|x\rangle \text{ (für alle } x \in \Sigma^n) \tag{3.17}$$

Zusätzlich wird noch ein ODER Gatter benötigt für n-Qubits. Ein klassisches ODER ist folgendes:

$$OR(x) = \begin{cases} 0 & x = 0^n \\ 1 & x \neq 0^n \end{cases} \quad \text{(für alle } x \in \Sigma^n) \tag{3.18}$$

Wenn ein einziges Bit $\neq 0$ ist, dann ist der Ausdruck korrekt. Ein ODER als Quantengatter sieht so aus:

$$Z_{OR} = \begin{cases} |x\rangle \ x = 0^n \\ -|x\rangle \ x \neq 0^n \end{cases} \quad \text{(für alle } x \in \Sigma^n\text{)} \tag{3.19}$$

Wenn ein einziges Qubit nicht $|0\rangle$ ist, dann wird von allen die Phase umgekehrt. Damit sind alle Gatter bekannt, die für Grover's Algorithmus benötigt werden. Die Grover operation ist selbst definiert als:

$$G = H^{\otimes n} Z_{OR} H^{\otimes n} Z_f \tag{3.20}$$

Die Notation $H^{\otimes n}$ in 3.20 bedeutet, anwendung des Hadamard-Gatters über n-Qubits. Um diese Gleichung als Algorithmus zu verstehen, muss man sie von rechts nach links lesen. Die Schritte des Grover's Algorithmus kann wie folgt beschreiben:

1. Initialisierung: n-Qubits werden mit $H^{\otimes n|0^n\rangle}$ initilisiert.
2. Iteration: Der Grover Operator G, wird t-mal angewandt.
3. Messung: Nach t iterationen, werden die Qubits gemessen, mit hoher Wahrscheinlichkeit auf Korrektheit gemessen.

Für t wird $\frac{\pi}{4}\sqrt{n}$ verwendet, warum es diese Anzahl ist, soll nicht darauf eingegangen werden.

Beispielhafte Anwendung

Abschließend ein Beispiel wie der Algorithmus schrittweise angewendet wird.

Es werden in einem Anfangszustand, n-Qubits als 0 initilisiert.

$$\psi_1 = |0^n\rangle \tag{3.21}$$

Die Qubits werden mit Hadamard-Gattern in eine Superposition gebracht.

$$\psi_2 = H^{\otimes n}|\psi_1\rangle \tag{3.22}$$

Nun wird der Grover Operator t-mal angewendet:

$$|\psi_{t+2}\rangle = G^t|\psi_2\rangle = \underbrace{G \circ G \circ \cdots \circ G}_{t\text{-mal}} \ |\psi_2\rangle \tag{3.23}$$

Dann erfolgt die Messung und mit hoher Wahrscheinlichkeit sollte der String gefunden worden sein, falls dieser existiert. für $t \approx \frac{\pi}{4}\sqrt{n}$ haben wir eine Laufzeitkomplexität von $O(\sqrt{n})$, es wird

vermutet dass es unmöglich ist, dieses Problem in weniger als $O(\sqrt{n})$ Schritten zu lösen [42]. Für dieses Problem gibt es auch randomisierten Algorithmen Algorithmus, der dieses Problem in $O(\sqrt{n})$ lösen kann.

3.2.6 Shor's Algorithmus

Wegen diesem Algorithmus besteht die Notwendigkeit der Post-Quanten-Kryptografie und folglich diese Bachelorarbeit. Shor's Algorithmus ist ein hybrider Algorithmus, teile davon werden klassisch berechnet und der rechenintensive Teil wird auf einem Quantencomputer ausgeführt, nämlich das unterliegende Problem der Periodenfindung. Es ist möglich herauszufinden, ob eine Funktion periodisch ist oder nicht. Wenn mit der Periode ω eine Funktion gesucht wird, für:

$$f(x + \omega) = f(x) \tag{3.24}$$

Dann kann shor's Algorithmus, effizient die Periode ω finden, wenn diese existiert. Aus der Gleichung-3.24 macht sich die Ähnlichkeit zu Simon's Problem sichbar, tatsächlich war Simon's Algorithmus auch eine Inspiration für Shor's Algorithmus.

Um grob zu verstehen, wie Shor's Algorithmus funktioniert, wird das finden von Primfaktoren am Bespiel von $N = 15$, $p = 3$ und $q = 5$ gezeigt. Angenommen wir wissen nur das $N = 15$ ist, p und q sind unbekannt. Als erstes wird eine Zahl a mit $1 < a < N$ gewählt. Wenn $gcd(a, N) > 1$ gilt, dann ist bereits ein nicht-trivialer Teiler von N gefunden, ansonsten muss ein r (hier für ω) berechnet werden, für das gilt:

$$a^r \equiv 1 \bmod N \tag{3.25}$$

r muss eine gerade Zahl sein und die Bedingung:

$$a^{r/2} \not\equiv -1 \bmod N \tag{3.26}$$

muss erfüllt sein. Dann liefern:

$$gcd\left(a^{r/2} - 1, N\right) \tag{3.27}$$

$$gcd\left(a^{r/2} + 1, N\right) \tag{3.28}$$

jeweils, nicht-triviale Teiler von N.

Wenn diese Bedingungen erfüllt sind, dann wissen wir auch dass $a^2 - 1 \equiv 0 \bmod N$ dann gelten muss und dass dadurch $a^2 - 1$ ein vielfaches von N ist. Deshalb gibt es auch eine ganze Zahl l,

für die gilt:

$$a^r - 1 = lN \tag{3.29}$$

l ist ein vielfaches von N. Nach der dritten Binomischen Formel, können wir es auch so umschreiben:

$$\left(a^{r/2} - 1\right)\left(a^{r/2} + 1\right) = lN. \tag{3.30}$$

Das bedeutet, es muss $p = gcd(a^2 - 1, N)$ und $q = gcd(a^2 + 1, N)$ gelten, oder umgekehrt.

Um den größten gemeinsamen Teiler zwischen zwei Zahlen, a und b, zu nennen, verwenden wir die Bezeichnung $\gcd(a, b)$.

Algorithmus 1 shor's Algorithmus

Require: ungerade, zusammengesetzte Zahl N
Ensure: Nichttrivialer Teiler von N

```
1:  repeat
2:      a ← zufällige Zahl im Bereich {2, ..., N − 1}
3:      d ← gcd(a, N)
4:      if d > 1 then                ▷ Falls nichttrivialer Teiler gefunden wurde, abgeschlossen
5:          return d
6:      end if
7:      r ← Ordnung von a modulo N                                          ▷ Quantenteil
8:      if r gerade und a^{r/2} ≢ −1 (mod N) then
9:          return gcd(a^{r/2} − 1, N)
10:     end if
11: until gcd(a^{r/2} − 1, N)
```

Der schwierige und rechenintensive Teil des Algorithmus ist ist in Zeile 7, die Ordnung r zu finden. Um dies effizient zu berechnen, ist ein Quantencomputer notwendig. Der Algorithmus zur Periodenfindung wird anschließend vereinfacht erklärt und warum das überhaupt auf die Findung der Ordnung angewandet werden kann.

Wenn man sich die Reihe der berechnungen von 2^n mod 15 anschaut, sieht man dass sich die

Werte in gleichen Abständen wiederholen.

$$2^0 \bmod 15 = \boxed{1}$$
$$2^1 \bmod 15 = 2$$
$$2^2 \bmod 15 = 4$$
$$2^3 \bmod 15 = 8$$
$$2^4 \bmod 15 = \boxed{1}$$
$$2^5 \bmod 15 = 2$$
$$2^6 \bmod 15 = 4$$
$$2^7 \bmod 15 = 8$$
$$2^8 \bmod 15 = \boxed{1}$$
$$\vdots$$

An einem Funktionsgrafen lässt sich das deutlicher sehen.

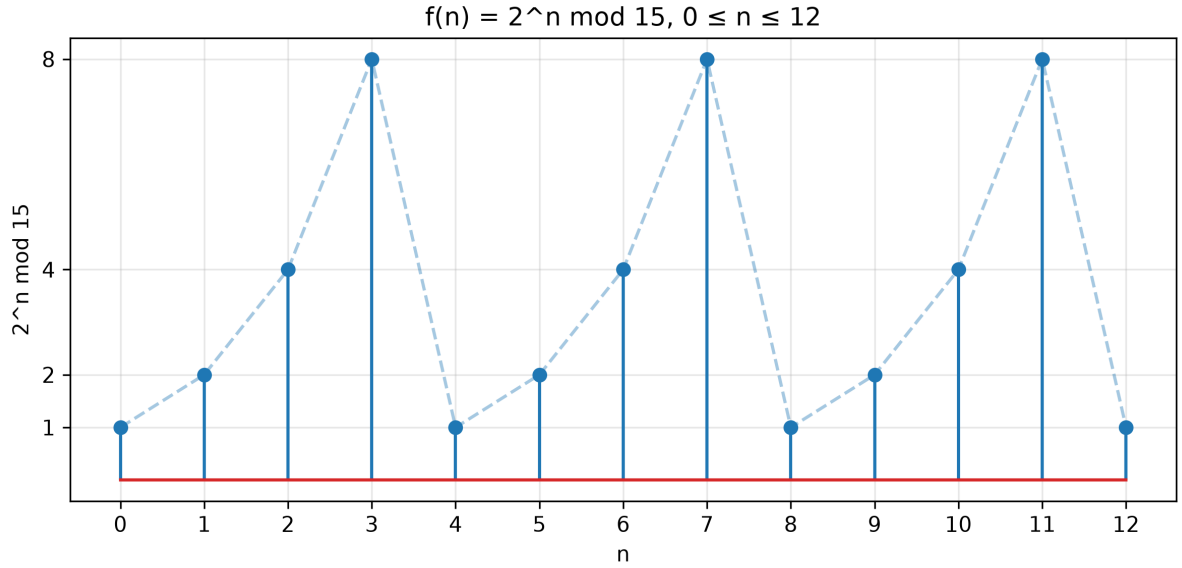


Abbildung 3.6: Funktionsgrafen von $2^n \bmod 15$, nach eigener Darstellung

Wie in Abbildung-3.6 zu sehen ist, weißt die Funktion ein Muster auf. Sie wiederholt sich nach einer bestimmten Anzahl an Schritten, dadurch lässt sich der Algorithmus der Periodenfindung ebenfalls auf sie anwenden. Die Funktion aus Abbildung-3.6, wiederholt sich immer nach 4 Schritten, dabei spricht man von einer Ordnung (r) von $r = 4$.

Der Algorithmus zur Periodenfindung, lässt sich mit IBM's Qiskit, folgendermaßen als Schaltkreis darstellen:

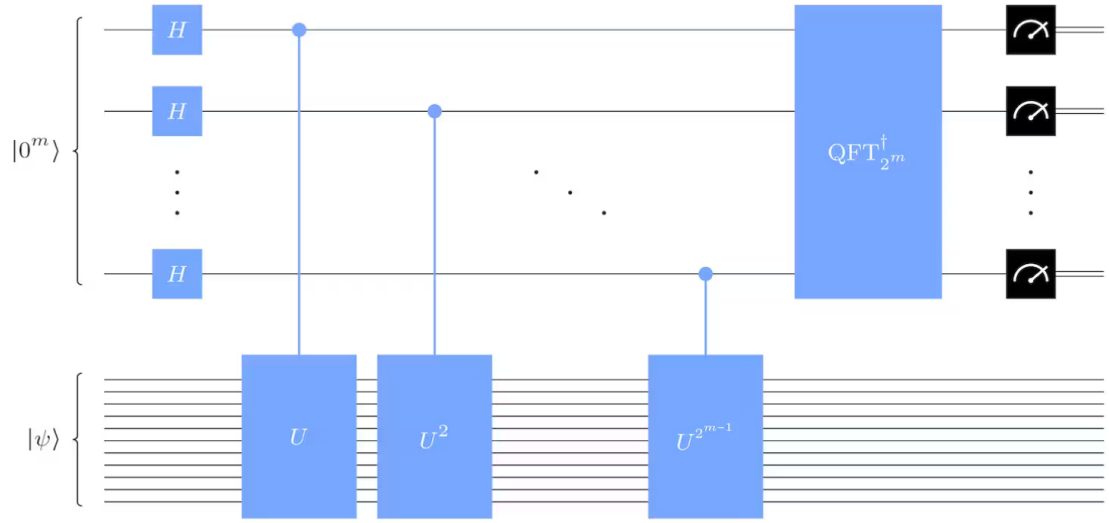


Abbildung 3.7: Schaltkreis für die Periodenfindung

In Abbildung-3.7 sind gleich einige Qubit Zustände zu sehen und Unitäre Operatoren, die vereinfacht der Reihe nach erklärt werden sollen. Die Qubits in $|0^m\rangle$ sind da für die Phasenabschätzung, durch die steigende Anzahl an verwendeten Qubits, steigt die genauigkeit der Phasenabschätzung. Die eigentlichen Zustände werden in $|\psi\rangle$ berechnet, durch die Parallelisierung von Quantenprozessoren kann eine sehr große Menge an Zuständen gleichzeitig berechnet werden.

der Operator U_a ist folgend definiert, dass dieser die modulare Berechnung ausführen kann:

$$U_a|x\rangle = |ax \bmod N\rangle \tag{3.31}$$

Für das Beispiel mit $a = 2$ und $N = 15$ wird die folge wie folgt berechnet:

$$\begin{aligned} U_2|1\rangle &= |2\rangle \\ U_2|2\rangle &= |4\rangle \\ U_2|4\rangle &= |8\rangle \\ U_2|8\rangle &= |1\rangle \\ &\vdots \end{aligned}$$

Im Anschluss wird eine Quanten-Fourier-Transformation auf die erste Register angwendet, durch diese lassen sich Messwerte erzeugen, aus denen sich die Periodenlänge r bestimmen lässt.

3.3 Post-Quanten-Kryptografie Signatur Algorithmen

Durch die mögliche Bedrohung von Quantencomputern in der Zukunft, werden neue Kryptographische Verfahren standartisiert. Das NIST hat dabei fünf Sicherheitsstufen definiert als Referenz welches Niveau für die Algorithmen zu erbringen ist. Die Sicherheitsstufen und deren Referenz sind in Abbildung-3.4 zu sehen[66].

Sicherheits Level	Vergleich	Angriffsverfahren	Ressistenz gegen Quantencomputer
1	$\geq$ AES128	vollständige Schlüsselsuche	schwach
2	$\geq$ SHA256	Kollisionsangriff	stark
3	$\geq$ AES192	vollständige Schlüsselsuche	stärker als Level 1 oder 2
4	$\geq$ SHA384	Kollisionsangriff	sehr stark
5	$\geq$ AES256	vollständige Schlüsselsuche	stärkste Ressistenz

Tabelle 3.4: NIST Sicherheitslevel für PQC-Verfahren

3.3.1 Der Umstieg von derzeitigen Digitalen Signaturen zu PQC

Die heutigen Digitalen Signaturen ((EC)DSA, (EC)ElGamal) basieren auf dem diskreten Logarithmus Problem oder bei RSA auf der schwierigkeit des faktorisierens großer Semi-Primzahlen in dessen Primfaktoren. RSA hat den Vorteil, dass die verschlüsselung und die Signatur sich nicht stark von einander unterscheiden.[54]

Die Verfahren aus dem NIST-Auswahlprozess haben längere Signaturen und nicht so einfach wie RSA. Der Signatur- und Verschlüsselungsalgorithmus unterscheiden sich stärker voneinander. Für DNSSEC sind nur die Signaturen interessant.

3.3.2 NIST Auswahlprozess für Digitale Signaturen

Zur Zeit dieser Arbeit befindet sich der NIST Auswahlprozess in der 4-ten Runde, nach dem Stand vom 11.März.2025. Als digitalen signatur Standart zugelassen sind momentan CRYSTALS-Dilithium (ML-DSA), FALCON (FN-DSA) und SPHINCS$^+$ (SLH-DSA) [20]. Zusätzliche Kandidaten für Digitale Signatur Schemas, die noch standartisiert werden sollen, sind noch in der evalution. Anfang der dritten Runde waren noch FAEST, HAWK, MAYO, MQOM, QR-UOV, SDitH, SNOVA, SQIsign, und UOV vorhanden [52].

Crystals-Dilithium

Das von der NIST favorisierte Verfahren, es wurde unter anderem von Deutschen Kryptographen entwickelt und basiert auf der schwierigkeit den nächsten Gitterpunkt zu einem Punkt in einem

Hochdimensionalem Gitter zu finden. Es wird ein Gleichungssystem aufgestellt für das eine Lösung mit kleinen Zahlen gefunden werden muss. Es basiert auf dem Short Integer Solution (SIS) Problem. Das Gleichungssystem, welches auch der öffentliche Schlüssel ist, enthält zudem kleine Fehler. Der private Schlüssel ist die Lösung dieses Gleichungssystems. Um aus der Einwegfunktion (aufstellen eines fehlerhaftes Gleichungssystems) ein valides Kryptosystem zu bauen, wird die sog. Fiat-Shamir Funktion angewandt. Wenn die Einwegfunktion sich auch mit Quantencomputern nicht effizient lösen lässt, ist das Verfahren durch die Fiat-Shamir-Transformation Quanten sicher.[60, 61]

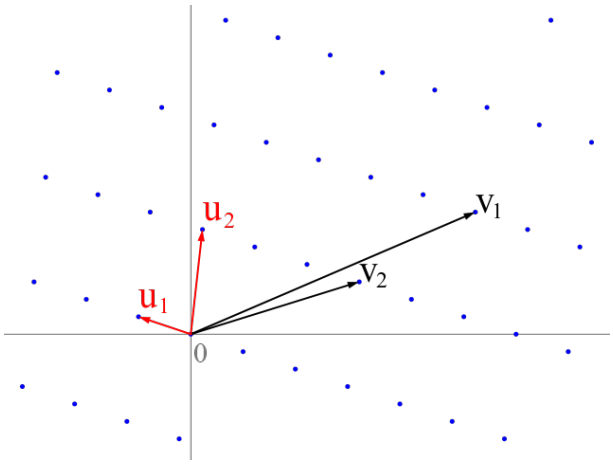


Abbildung 3.8: 2-dimensionales Gitter mit einem Punkt und dem nächstgelegenen Gitterpunkt [28].

Abbildung 3.8 zeigt das SIS-Problem in einem zweidimensionalen Gitter. Von dem Punkt V_1 aus soll der nächstgelegene Gitterpunkt V_2 gefunden werden, die Basisvektoren (u_1, u_2) stellen eine schlechte Basis bereit. Es muss ein höherdimensionales Gitter und eine schlechte Basis verwendet werden, damit das Problem schwer genug ist[61].

FALCON

Fast-Fourier **L**attice-based **Co**mpact Signatures over **N**TRU, kurz **FALCON**. Das einzige dieser Kryptosystem, welches mit Gleitkommzahlen rechnet. Es basiert auch auf dem Short Integer Solution (SIS) Problem über NTRU Gittern [55]. FALCON gilt aufgrund von Gleitkommazahlen, rekursiven Funktionen und mehreren diskreten Gaussverteilungen als schwerig zu implementieren. FALCON hat den Vorteil, dass es kleine Signaturen und öffentliche Schlüssel hat und das es schneller im verifizieren ist. Unter anderem nutzt es die Fast-Fourier-Transformation mit einer Superlinearen Laufzeitkomplexität von $O(n \log n)$ anstelle der Fourier-Transformation mit einer quadratischen Laufzeitkomplexität $O(n^2)$.[53, 67] Crystals-Dilithium und FALCON nutzen zur Verifikation „Fiat-Shamir with aborts“, welches ein challenge-response Verfahren ist[6].

SPHINCS$^+$

Ein konservatives Verfahren, welches auf der Sicherheit von Hashfunktionen aufbaut. Da Hashfunktionen gut erforscht sind und nicht anfällig sind durch Shor's Algorithmus und Grover's Algorithmus die Sicherheit nur bedingt beeinträchtigt, gelten sie als ziemlich sichere Alternative. SPHINCS$^+$ ist eine modifizierte Version von SPHINCS und ein sogenanntes „zustandloses Signaturverfahren". Es unterstützt die Hashfunktionen, SHAKE und SHA, zudem in unterschiedlichen Längen und als short oder fast. Intern funktioniert dieses Verfahren auch mit Merkle-Trees und dem HORST-few-times Signaturverfahren, dabei werden auch Schlüssel zufällig gewählt. Hashbasierte Verfahren haben den Vorteil dass sie, wie in Abbildung-3.2 zu sehen ist, kurze öffentliche Schlüssel, aber dafür sehr lange Signaturlängen haben [66, 26, 31, 41].

3.3.3 weitere PQC-Signatur-Verfahren

Rainbow

Rainbow ist ein multivariates Polynomiales Kryptosystem, welches eine Generalisierung des Oil and Vinegar scheme (UOV) ist. Das unterliegende Problem ist, dass ein multivariates Gleichungssystem von Polynomen über einem endlichen Feld ein NP-schweres Problem ist. Ein Gleichungssystem wie das folgende (aus serious cryptography[26]) mit vier unbekannten ist gegeben, um es zu lösen müssen alle vier unbekannten x_1, x_2, x_3, x_4 gefunden werden.

$$
\begin{aligned}
x_1x_2 \;+\; x_3x_4 \;+\; x_2 \;&=\; 1 \\
x_1x_3 \;+\; x_1x_4 \;+\; x_2x_3 \;&=\; 12 \\
x_1^2 \;+\; x_3^2 \;+\; x_4 \;&=\; 4 \\
x_2x_3 \;+\; x_2x_4 \;+\; x_1\,x_4 \;&=\; 0
\end{aligned}
$$

Solch ein Gleichungssystem zu lösen ist NP-schwer und lässt sich auch mit Quantencomputern nicht effizient lösen. Multivariate Kryptosysteme wurden bereits öfters gebrochen, trotz der Annahme sicher zu sein und die Berechnungen sich oft auch ressourcen lastig, auch wenn die Signaturen eher kurz sind, was dazu führt, dass der Einsatz dieser Systeme in der Praxis recht selten ist[26]. Das Rainbow kryptosystem wird beschrieben als eine mehrschichtige oil-and-vinegar konstruktion und dessen generalisierung[39]. Rainbow wurde im Laufe des NIST-Verfahrens vollständig gebrochen und soll nicht mehr verwendet werden. Es wurde gezeigt, dass sich der private Schlüssel durchschnittlich innerhalb 53 Stunden berechnen lässt [32].

HAWK

Ebenfalls ein Algorithmus der auf dem SIS-Problem aufbaut, genauer gesagt dem Gitter isomorphismus problem, der Algorithmus basiert sogar auf FALCON. Hawk sein privater Schlüssel ist selbst eine Matrix und es nutzt als Signatur und zur Verifikation zusätzlich einen salt [59]. Das Unternehmen Anthropic hat mit einer angepassten version ihres large language models „Mythos 6“ (vor offiziellem Release) einen Angriff auf dieses Kryptosystem gebaut, welches innerhalb von 60 Stunden eine signifikaten Verbesserung des bisher besten Angriffs gefunden hat. Um dieses Projekt durchzuführen, wurde eine Sandbox Umgebung eingerichtet mit python, Sage und Zugang zu kryptografischen Publikationen, überwacht von einem menschlichen Operator mit einem Hintergrund in Theorethischer Informatik. Nach 60 Stunden und Kosten von 100.000$ in Token, hat das System dessen Ergebnisse mittels einer verfikations pipeline verfiziert und dessen Ergebnisse präsentiert. Die bisher angenommene Sicherheit von HAWK-256 von 2^{64} lies sich auf 2^{38} reduzieren, das Team von HAWK spricht von einer ungefähren halbierung der Blockgröße um einen equivalenten privaten Schlüssel zu berechnen. Nach der entdeckung des Angriffs durch Anthropic wurde HAWK vom NIST Auswahlprozess zurückgezogen, weil HAWK kein konkurent mehr ist für derzeitige Gitter-basierte Kryptosysteme, trotz dass Anthropic diesen Angriff als „unpraktisch für größere Schlüssel“ beschrieben hat [21, 4, 68].

SIKE

Basierend auf Elliptischen Kurven und angenommen Quantensicher zu sein, wurde dieses Verfahren dem NIST Auswahlverfahren eingereicht. Nach kryptoanalytischer Betrachtung haben sich doch Schwächen gezeigt, sogar so fatal, dass einige Angriffe es ermöglichen das Verfahren innerhalb weniger Minuten komplett zu brechen. Das Verfahren in irgendeiner Sicherheitsrelevanten Sicht zu verwenden ist absolut ausgeschlossen [34, 12].

SNOVA

Ein Multivariates Kryptosystem, welches auf dem Unbalanced Oil and Vinegar (UOV) Schema aufbaut. Dadurch dass es über nicht-kommutativen Ringen rechnet, soll es im gegensatz zu OUV hat es kürzere Schlüssellängen haben effizienter rechnen. SNOVA's Schlüssellängen für den öffentlichen Schlüssel und für die Signatur sind sogar kürzer als die von FALCON [14, 59].

UOV

Das älteste der multivariaten Verfahren aus dem NIST Auswahlprozess und die Basis für viele Verfahren dieser Art, wie SNOVA, MAYO oder Rainbow. Die Falltür Information dieses Verfah-

ren basiert auf einem linearen m-dimensionalem untervektorraum, welcher als oil-space bezeichnet wird. Der öffentliche Schlüssel selbst besteht aus m-Matrizen [59]. Der öffentliche Schlüssel ist riesig mit 412kB und unbrauchbar für DNSSEC.

CROSS

Codes and Restricted Objects Signature Scheme (CROSS) baut auf Zero-Knowledge (ZK) Beweisen auf. Durch eine Fiat-Shamir Transformation auf einen nicht-interaktiven ZK-Beweis. Das unterliegende Problem ist das NP-vollständige „restricted syn-drome decoding problem“. Es ist ein code-basiertes Sicherheitssystem, welches intern Merkle-Trees verwendet [27, 59]. Es wurde entworfen um schnell zu sein, für DNSSEC ist es nicht optimal durch große Schlüsselgrößen.

MQOM

MQ on my Mind (MQOM). Wie schon im Namen steht, ist es ein Verfahren welches auf der Schwierigkeit von multivariaten Gleichungssystemen (MQ) aufbaut. Es baut auf Zero-Knowledge Beweisen auf und wird durch die Fiat-Shamir Transformation zu einem digitalen Signatur Verfahren umgewandelt. Im ist es ein MQ-Verfahren, im Gegensatz zu CROSS, welches ein code-basiertes Verfahren ist [30]. CROSS und MQOM unterscheiden sich signifikant, auch wenn sie beiden mit ZK und Fiat-Shamir funktionieren.

MAYO

Ein Verfahren welches ebenfalls auf UOV aufbaut. Es wurde entworfen als ein multivariates Kryptosystem, welches nach UOV funktioniert, aber kürzere Schlüssellängen hat für den öffentlichen Schlüssel und die Signatur [70].

3.4 Proof-of-Concept: Performance testing der PQC-Algorithmen

Als Proof-of-Concept wurde mit der open-source Bibliothek liboqs (`https://github.com/open-quantum-safe/liboqs`) pqc in den DNS Server powerDNS (`https://github.com/PowerDNS/pdns`) integriert und dessen Performance gemessen. Die Implementierung dieser des PoC und des benchmarkings wurde mit Hilfe des LLM Kimi-K3 geschrieben [18]. Das PoC ist ein Fork des powerDNS Servers. Es wurde eine neue Datei „oqssigners.cc“ erstellt, sie bietet ein template für die PQC-Algorithmen. Die Datei dnssec.hh wurde um die enum-Werte 240-245 ergäänzt. Die Dateien dbdnsseckeeper.cc und dnssecinfra.cc wurden angepasst, damit die Bits-default (128, NIST-Level 1) verfügbar sind. Noch zusätzlich wurden weitere Autotools hinzugefügt um die verwendung der

neuen Algorithmen zu ermöglichen. Es wurden keine angepassten NSEC-Versionen implementiert, das PoC wurde auf die einfache Erweiterung der Signaturalgorithmen beschränkt.

Die Signaturverfahren von DNSSEC wurden um folgende erweitert:

Algorithmus	ID	Security (Bit)	Länge öffentlicher Schlüssel (Byte)	Länge Signatur (Byte)
FALCON512	240	128	897	656
MAYO1	241	128	1456	464
SNOVAIK	242	128	376	528
MQOM3GF16	243	128	64	3316
SLHDSASHA2128S	244	128	32	7856
CROSSRSDP128	245	128	77	13152

Tabelle 3.5: PQC-Algorithmen im PoC

Die ID-Nummern wurden nach den freien Nummern der IANA gewählt. Das UOV-Verfahren kann nicht praktisch verwendet werden, weil der public key den Rahmen eines DNSKEY-Records (RDATA-Maximum: 65535 Byte) sprengt mit seinen 412KB. Manche dieser Verfahren liegen, mit der göße des öffentlichen Schlüssels, noch in der größenordnung, welches UDP zulässt, wie FALCON. Für CROSS z.B. ist TCP-fallback notwendig. Für das PoC wurde die minimalen Sicherheitsniveaus verwendet, die annahme ist, dass der Time-to-Live (TTL) Wert für einen DNSKEY, wie in Abbildung-3.1 von unter einer Stunde, einem Angriff standhalten sollte. Auch die geringste Sicherheitsstufe sollte bei einem modernen Kryptografischen Algorithmus hier standhalten.

Das PoC, das benchmarking und die Auswertung des Benchmarking sind auf Github unter folgenden Links zu finden:

- https://github.com/MarkPublicRepo/powerDNS-PQC.git **Proof-of-Concept PowerDNS Server fork**
- https://github.com/MarkPublicRepo/powerDNS-PQC-Benchmark.git **Benchmarking**
- https://github.com/MarkPublicRepo/powerDNS-PQC-Benchmark-auswertung.git **Benchmarking Auswertung**

3.4.1 Benchmarking Auswertung

Durch benchmarking der einzelnen Algorithmen, eines Ressource Record, für jeweils keypair, signieren und verifizieren für 100 iterationen, haben sich folgende Messwerte ergeben. Zu beachten ist, dass wegen der starken Unterschiede in den Messergebnissen, für jede Auswertung eine logarithmische Skala erforderlich ist.

In Abbildung-3.6 lässt sich ablesen, dass die Laufzeit für die klassischen Verfahren die deutlich längste Laufzeit für keypair haben. Ebenso haben sie eine signifikant höhere Standartabweichung

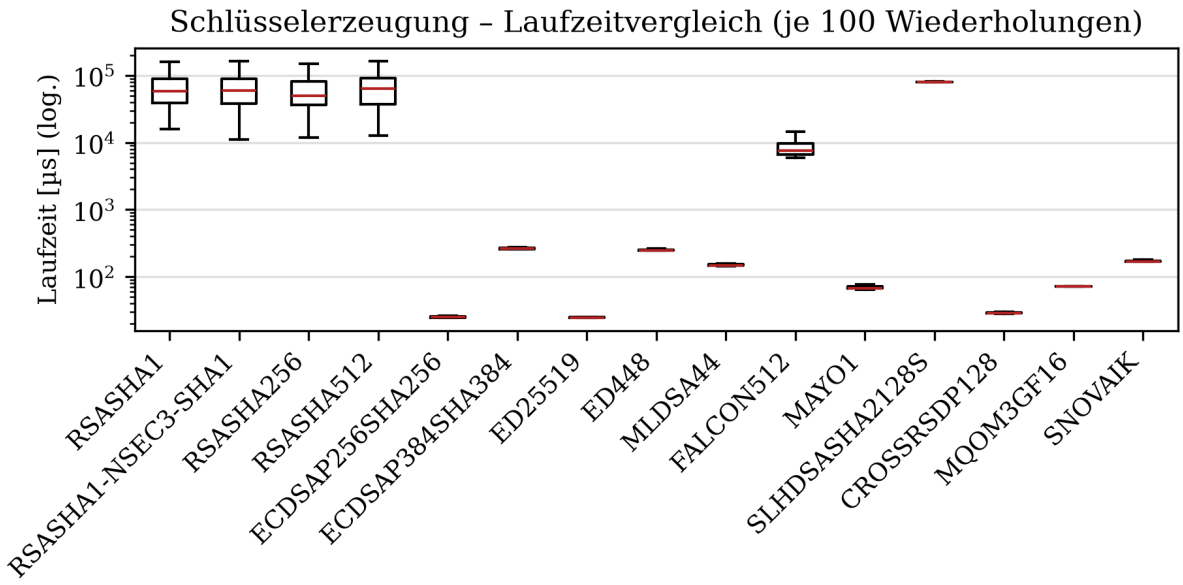


Tabelle 3.6: Auswertung von keypair im Laufzeitvergleich

im Gegensatz zu den PQC-Signaturverfahren. Die längste Laufzeit hat allerdings SLHDSASHA2128S bei Keypair/Schlüsselerzeugung, mit 71496.01 µs im Mittel σ^2 gefolgt von RSASHA512 mit 71496.01 µs. Was sich anhand der Boxplots gut ablesen lässt, ist das die klassischen Verfahren auch im unteren Berecih noch neben SLHDSASHA2128S die längsten Laufzeiten haben. Mit Abstand hat danach, hat FALCON512 die längste Laufzeit. Die Schnellsten Verfahren sind die Elliptische Kurven Varianten und der schnellste ist ED25519, was nicht überrascht, da dieser extra auf geschwindigkeit ausgelegt ist.

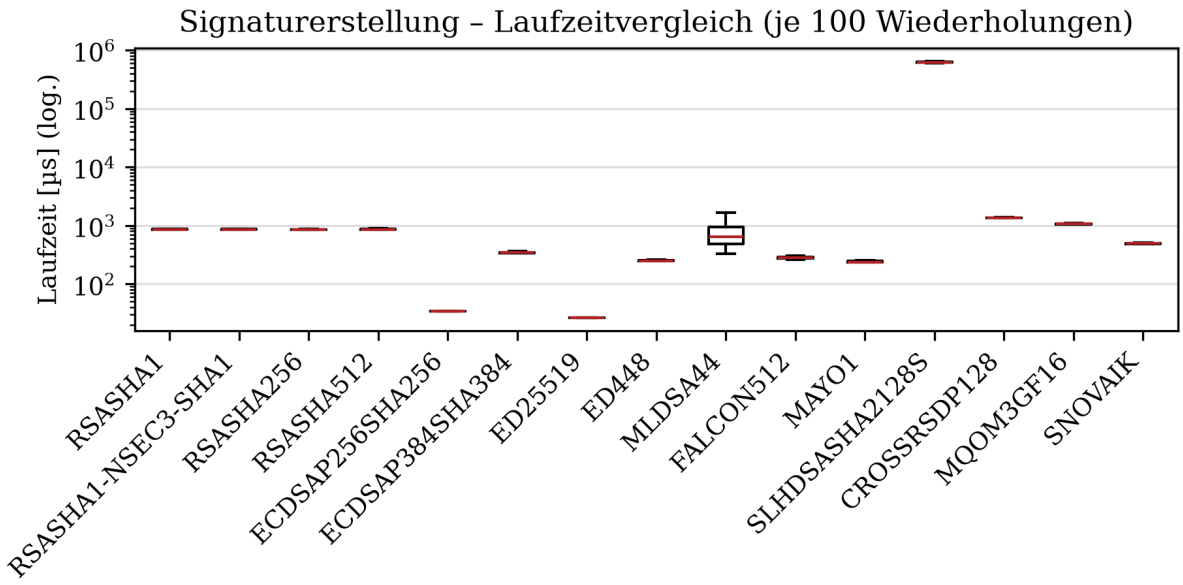


Tabelle 3.7: Auswertung der Signaturerstellung im Laufzeitvergleich

Auf den ersten Blick lässt sich in Abbildung-3.7 die deutlich geringere Streuung im Gegensatz zu Abbildung-3.6 sehen. Bis auf ML-DSA44 liegt hier nur geringe Streuung vor. Unter den Post-quanten Verfahren sind , FALCON512, MAYO1, SNOVAIK, teilweise ML-DSA44 und MQOM3GF16 (annähern) so effizient wie die RSA Varianten. Ebenso wie bei der Schlüsselerzeugung, siehe Abbildung-3.6, ist SLHDSASHA2128S, exponentiell langsamer als die restlichen Verfahren.

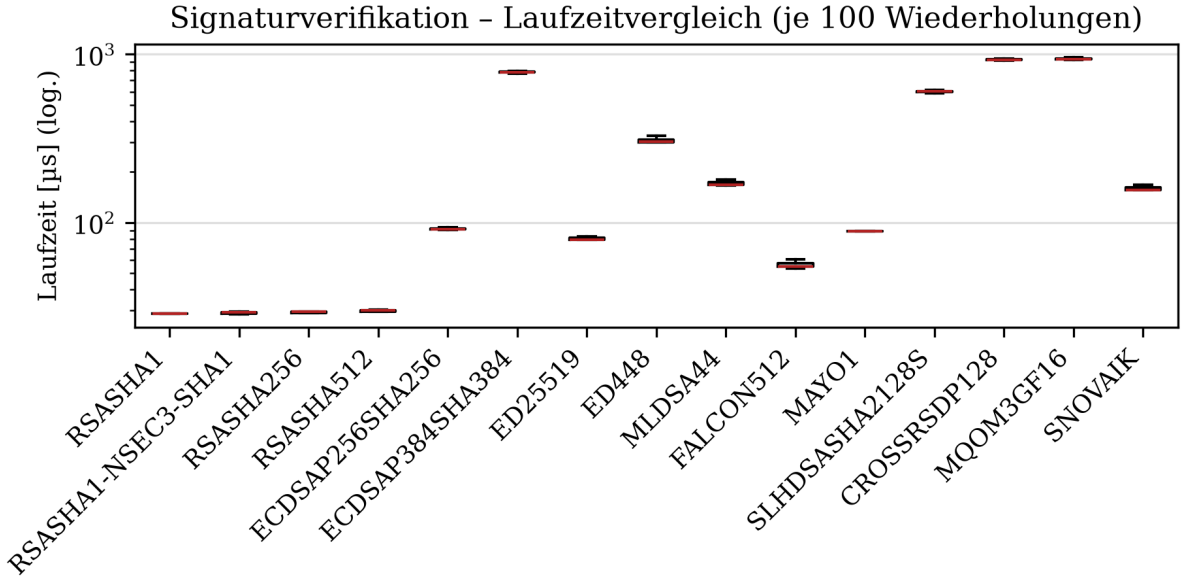


Tabelle 3.8: Auswertung der Verifikation im Laufzeitvergleich

Was Signaturverifikation angeht ist RSA weit vorne. Die beste Alternative ist FALCON, welches immer noch exponentiell schlechter ist, mit deutlich höherer Standartabweichung. Die Ellip-

tischen Kurven Verfahren sind hier vergleichsweise deutlich langsamer als zur Schlüsselerzeugung und Signatur.

Für DNSSEC speziell sind nicht alle Laufzeiten gleichwichtig. Einen Schlüssel muss der Herausgeber selbst erzeugen, er ist schließlich die „Person“, die dessen Identität beweisen muss. Ebenso ist das Signieren etwas, das durch offline signing geschehen kann. Was wirklich wichtig für den DNS service ist, ist dass die Signaturen effizient von DNS Dienst verfiziert werden können. Ebenso sind auch die Größen von privatem/öffentlichem- Schlüssel und Signatur. In dieser Hinsicht ist SLHDSASHA2128S in jeder Hinsicht der schlechteste Wahl, hashbasierte Verfahren haben den Vorteil dass sie auf der bekannten Sicherheit von Hashverfahren basieren[26], sind aber für DNSSEC der falsche Einsatz.

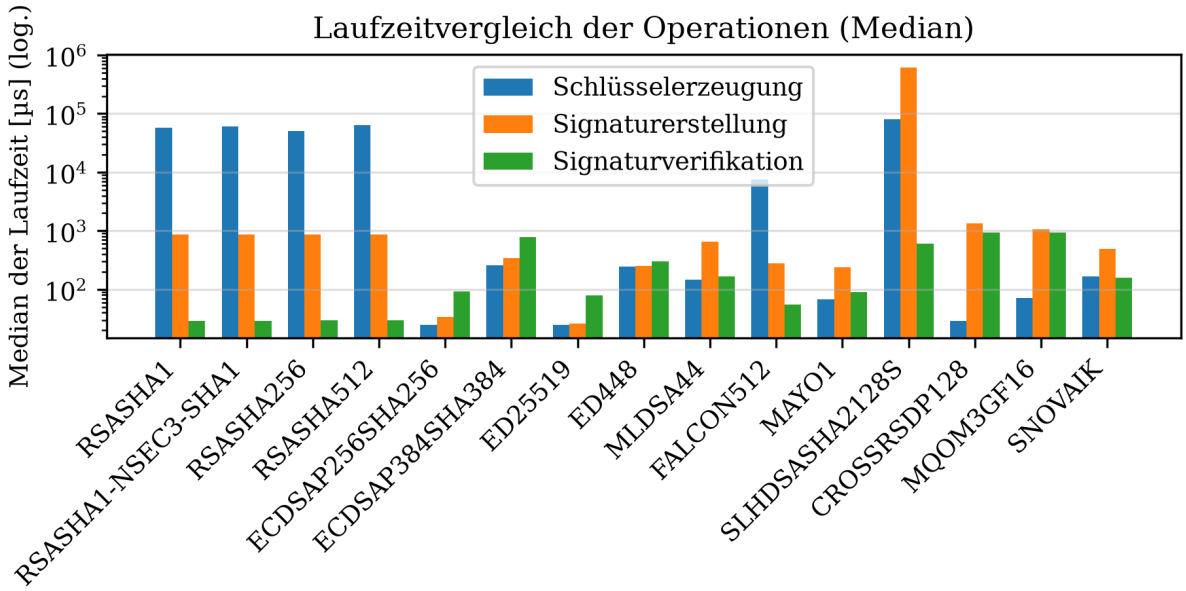


Tabelle 3.9: Median der unterschiedlichen Algorithmen

Trotz dass SLHDSASHA2128S konsistent in Schlüsselerzeugung, Signieren und in Verifikation ist, sind die Laufzeiten im Vergleich zu allen anderen viel schlechter, wenn nicht die größe der Schlüssel den Algorithmus als Wahl für DNSSEC im vornherein ausschließt. CROSSRSDP128 und MQOM3GF16 liegen bei der Laufzeit für Verifikation stehen sogar noch schlechter als SLHDSASHA2128S da. Unter berücksichtigung der Schlüssel/Signatur- größen, der priorisierung von signaturerstellung und besonders der Signaturverfikation, sind die Algorithmen, die noch in Frage kommen:

- ML-DSA44
- FALCON512
- MAYO1

- SNOVAIK

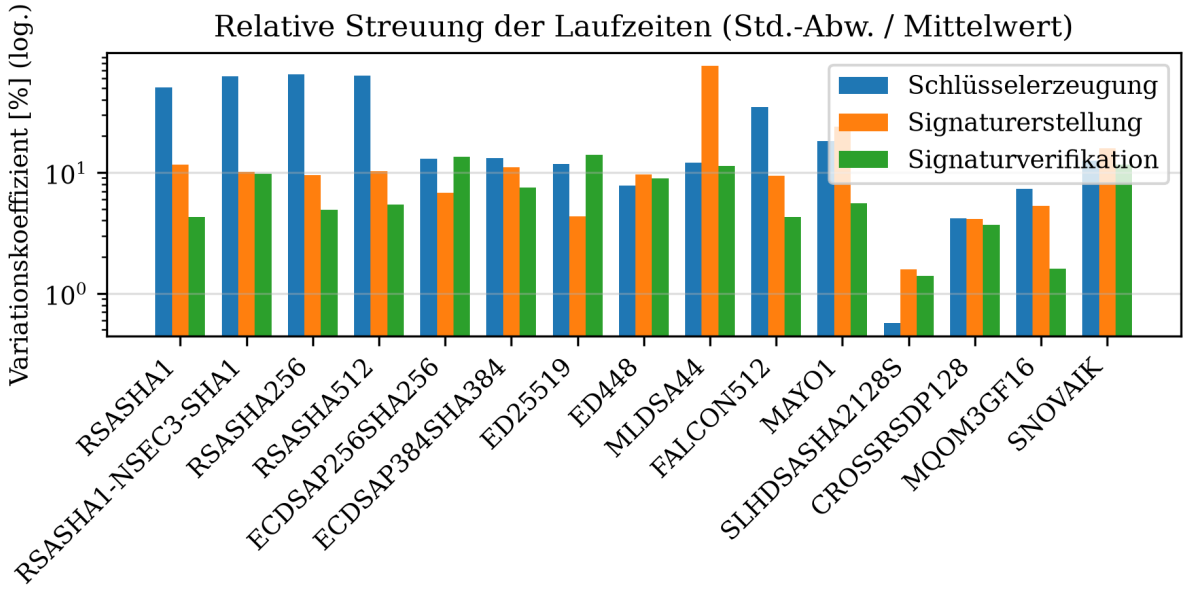


Tabelle 3.10: Variation der unterschiedlichen Algorithmen

Die Standartabweichung ist wichtig zu betrachten. Falls ein Algorithmus im Median effizient ist, aber dazu neigt stark auszuschlagen, kann das häufig Performance kosten verursachen. Das ist die einzige im PoC betrachtete Kategorie, in der SLHDSASHA2128S gute Messwerte zeigt, anhand der anderen Messwerte, ist dieser aber bereits als Kriterium ausgeschlossen. Die PQ-Signaturverfahren, die noch zu berücksichtigen sind, ML-DSA44, FALCON512, MAYO1 und SNOVAIK, haben folgende Ergebnisse.

ML-DSA44 hat zur Signaturerstellung deutlich die höchte Abweichung, die Verfikation gehört ebenso zu den höchsten. Wie in Abbildung-3.10 deutlich wird, hat ML-DSA44 auch deutlich die höchte Abweichung, nicht zur unter post-quanten Verfahren. Dennoch ist auch die maximale Abweichung noch totellierbar. Ebenso ist die Verfikation die höchste Standartabweichung unter den vier. Dennoch akzeptabel, aus Abbildung-3.7 lässt sich entnehmen, dass auch der worst-case noch gut ist.

FALCON512 hat die geringste Abweichung in der Signaturerstellung und in Verfikation. FALCON512 ist somit nicht nur das Verfahren, welches die geringe Performance einbüsungen bringt, sondern mit Signatur- und Schlüsselgröße lässt es sich auch weiterhin über UDP nutzen.

MAYO1 liegt unter den vier Verfahren zwischen ML-DSA44 und FALCON512 was Abweichung angeht. Mit vergleichweise kurzen Schlüssel- und Signaturlängen, ist es eine möglicherweise eine interessante alternative zu klassische Verfahren.

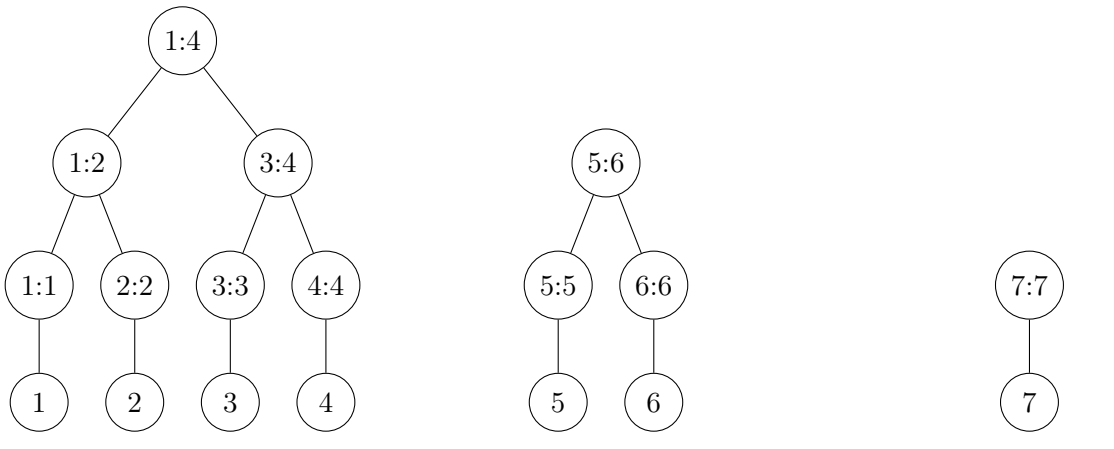
SNOVAIK hat im Vergleich zu FALCON512, geringe Laufzeiten aber dafür mehr Abweichung, bis auf Schlüsselerzeugung, was aber für DNSSEC nicht so wichtig ist.

3.5 Merkle Tree Ladder (MTL) Modus

Anstelle davon jede Nachricht einzeln zu signieren, kann man die Nachrichten auch als, fortschreitende Folge von Nachrichten betrachten. Das ist die Idee des MTL-Modus. Nicht nur PQC-Signaturen wie SLH-DSA oder CROSS verwenden intern Merkle-Trees sonders wie lassen sich auch in sonstiger Software effizient einsetzen. Im Fall von DNSSEC, lässt sich damit ein effizientes Signatur Schema erstellen.

Nicht jede neue Nachricht wird mit einem PQC-Algorithmus wie ML-DSA, FN-DSA oder SLH-DSA signiert. Es werden Nachrichten, jeh nach deren Anzahl in einem Baum „Rung“ angeordnet. Die Bäume ergeben zusammen die „Ladder“. Diese Ladder wird dann mit einem Algorithmus wie ML-DSA signiert. Der Baum verwendet für Nachrichten eine PQC-sichere Hashfunktion mit dieser die Knoten der Nachrichten signiert werden.[40] Die „Ladder“, die aus den einzelnen Bäumen besteht, wird dann nicht nach jeder neuen Nachricht signiert, sondern nur nach einer bestimmten Anzahl an Nachrichten wird der Rung mit dem PQC-Algorithmus neu signiert. Insgesamt verringert das den Aufwand zum Signieren, durch das hashen deutlich und es bleibt Quantensicher.

Folgendes Beispiel, behandelt den Fall, dass es sich um sieben Nachrichten handelt. Die Anzahl der Nachrichten wird in Binär umgewandelt, die Binärstellung gibt an wie viele Bäume es sind und wie groß sie sind. Als Beispiel werden sieben Nachrichten in einen Baum zusammengeführt. Durch die Umwandlung in Binär lässt sich ablesen $7_{10} = 111_2$, es sind drei Bits gesetzt, demnach gibt es drei Bäume. Das Höchstwärtige Bit ist vier, demnach enthält dieser Baum vier Elemente, der nächstgrößte Baum hat zwei Elemente und das letzte Bit gibt ein Element an. Für sieben Nachrichten sehen die Bäume, nach eigener Darstellung, folgendermaßen aus:



(a) Baum 100 (b) Baum 010 (c) Baum 001

Abbildung 3.9: Binärbaume des Merkle-Baumes für sieben Nachrichten

Unterstützte Hash Funktionen sind aus der SHAKE oder SHA2 Familie.[43] Es werden folgende

Hashes h für die Knoten und Blätter mit der Hashfunktion H berechnet:

$$h_1 = H(1);\ h_2 = H(2);\ h_3 = H(3);\ h_4 = H(4);\ h_5 = H(5);\ h_6 = H(6);\ h_7 = H(7) \tag{3.32}$$

$$h_{1:2} = H(h_1||h_2);\ h_{3:4} = H(h_3||h_4);\ h_{5:6} = H(h_5||h_6);\ h_{7:7} = H(7) \tag{3.33}$$

[37]

Die Ladder Λ_7 sieht so aus:

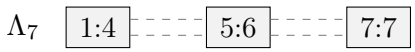


Abbildung 3.10: Ladder für sieben Nachrichten

Die Ladder wird dann mit einer PQC-Signatur σ wie ML-DSA oder SLH-DSA signiert. Die Signatur $\sigma(\Lambda_N)$ wird nach N Nachrichten ausgeführt, nicht nach jeder einzelnen, weil dann wäre keine Performance verbesserung vorhanden. Wenn eine weitere Nachricht aufgenommen wird, erhöht sich die Gesammtzahl auf acht. Dann ändert sich die Baumstruktur mit $8_{10} = 1000_2$ folgendermaßen:

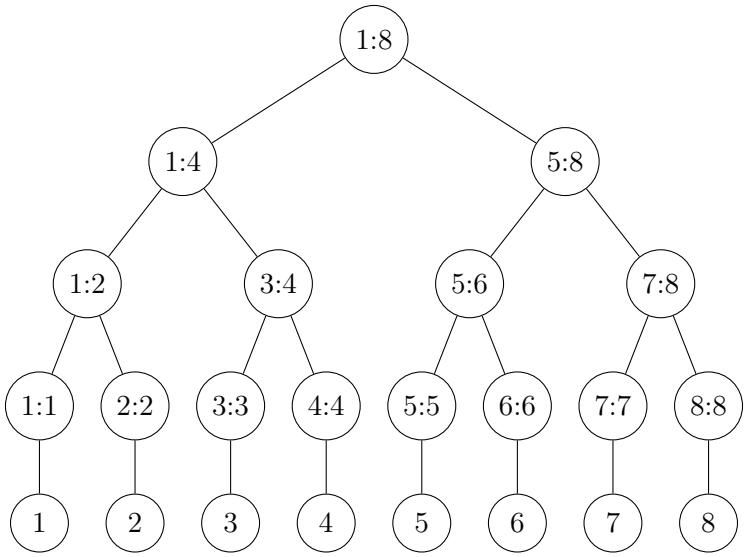


Abbildung 3.11: Binärbaum für acht Nachrichten

Hashes werden berechnet:

$$h_8 = H(h_8) \tag{3.34}$$

$$h_{7:8} = H(h_8);\ h_{5:8} = H(h_{5:6}||h_{7:8})\ h_{1:8} = H(h_{1:4}||h_{5:8}) \tag{3.35}$$

Die Ladder Λ_8 sieht nun so aus:

Die Hashes müssen nicht alle neu berechnet werden, nur diejenigen die betroffen sind. Die Berechnungen der Hashes und der Signatur sind hier vereinfacht dargestellt, das Prinzip ist aber



Abbildung 3.12: Ladder für acht Nachrichten

dass durch zusätzliche Nachrichten nicht alles neu berechnet werden muss und die Berechnungen der Hashfunktionen sind performanter als die Signatur.

Die Merkle-Tree-Ladder besteht aus einem Wald von Binärbaumen, die Baume können aber nicht genau die Domänen abbilden. Folgend, ein Beweis durch wiederspruch, dass eine Domäne nicht immer durch einen Merkle-Tree abgebildet werden kann.

Satz 1. *Ein gültiger DNS-Domänenbaum kann immer durch einen Merkle-Tree abgebildet werden.*

Beweis. Angenommen, die Aussage des Satzes gelte. Dann müsste jeder DNS-Domänenbaum einem Merkle-Tree (Rung) entsprechen. Ein Baum eines Merkle-Trees, auch „Rung“ genannt, ist entweder ein perfekter Binärbaum oder ein einzelnes Blatt. Um ein perfekter Binärbaum zu sein, gilt ein Baum hat $|V|$ Knoten und $|V| = 2^{n+1} - 1$, mit $n \in \mathbb{N}_0$ als höhe[7, 47].

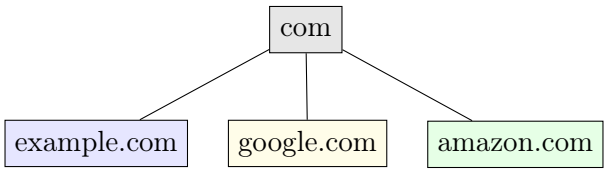


Abbildung 3.13: Gültiger DNS-Domänenbaum einer Domäne/Wurzel mit drei labels/Kindern

Ein gültiger DNS-Domänenbaum in Abbildung-3.13 mit $n = 1$ und $|V| = 4$, somit $|V| \neq 2^{n+1} - 1 = 3$.

Der Baum besteht nicht aus einem einzelnen Blatt: $|V| = 4 \neq 1$.

Somit ist der Baum in Abbildung-3.13 kein gültiger Rung. □

Die Struktur der Merkle-Tree-Ladder entsteht nach der Anzahl von Nachrichten, die darin liegen, sie entspricht nicht der der (Sub-) Domäne.

4 Fazit und Ausblick

Laut IBM soll die Quantencomputer-Branche bis 2035 schätzungsweise ein Volumen von 1,3 Billionen US-Dollar erreichen [65]. Michio Kaku beschreibt das kommen der Quantencomputer als ein potentiell größeres historisches Ergeignis als die Einführung der digitalen Computer [33]. Der aktuelle Entwicklungsstand eines kryptografisch relevanten Quantencomputer ist nach BSI immer noch auf dem Stand des NISQ-Zeitalters. NISQ ist eine Komplexitätsklasse von Quantenalgorithmen [35]. Die Quantencomputer welche innerhalb dieser Klasse rechnen, besitzen eine mittlere Anzahl an Qubits und sind nicht fehler korrigierend. Längere Rechenzeiten führen zu fehlern in Berechnungen. Die Entwicklung derzeitige Quantencomputer basiert derzeit mehr auf Heuristiken als auf mathematischen Auswertungen, wie in der Grafik von Abbildung-4.1 zu sehen ist.

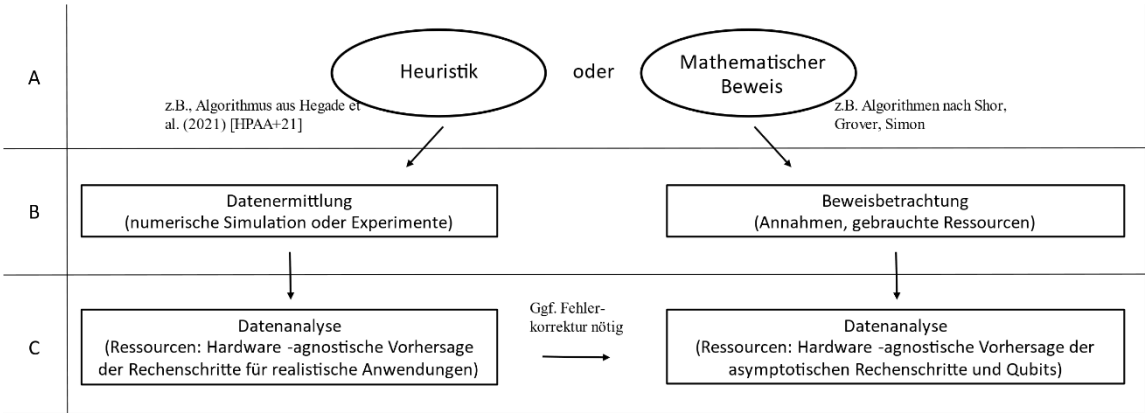


Abbildung 4.1: Entwicklungsstand Quantencomputer nach BSI [3]

Zur Zeitspanne bis zu einem Kryptoanalytisch relevanten Quantencomputer schreibt das BSI, dass Zukunftsprognosen oft genau 20 Jahre vor Eintritt vorausgesagt werden und durch neue Entwicklungen im Bereich der qLDPC-Codes könnte sich der Bereich sogar noch weiter, bis auf etwa 10 Jahre verkürzen [3].

Wenn im optimistischen Fall, von 10 Jahre bis zu einem Kryptoanalytisch relevanten Quantencomputer ausgegangen wird, ist die Notwendigkeit von PQC, bitter notwendig. Bereits in der Gegenwart ist das Sammeln von verschlüsselten Nachrichten relevant, da dies im Fall von Quantencomputern in einigen Jahren entschlüsselt werden könnte. Für DNSSEC speziell, ist das

nicht der Fall, da die Signaturen in geringeren Zeitabständen erneuert werden, also collect-now-forge-later tritt hier nicht ein.

Das Unternehmen Gardner spricht sogar von einem Zeitraum von fünf bis zehn Jahren bis zu Fehlerkorrigierenden Quantencomputern.

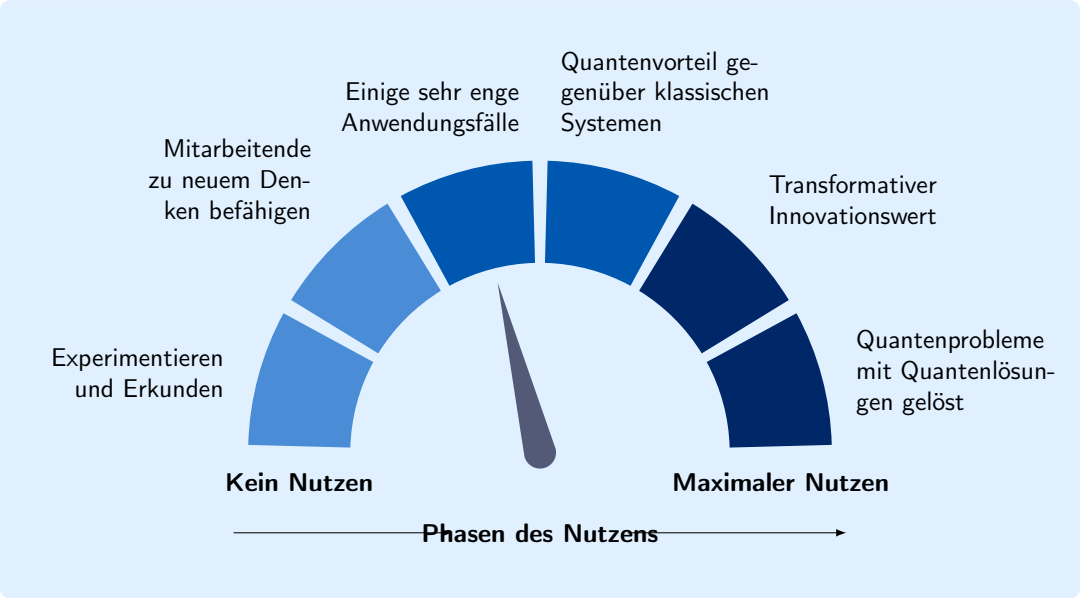


Abbildung 4.2: Geschäftlicher Nutzen von Quantencomputing über die Zeit. Eigene Darstellung nach [15].

In Abbildung-4.2 werden die Schritte der Entwicklung von Quantencomputern (nach Gardner) bildlich dargestellt. Momentan haben Quantencomputer nur einen sehr eingeschränkten Nutzen. In nur wenigen, angepassten Fällen haben sie einen Vorteil gegenüber klassichen Computern, wobei sich das hinsichtlich fortschritten in der Fehlerkorrektur bald ändern kann.

Diese Prognosen sind sehr optimistisch hinsichtlich der Entwicklung. Kritiker hingegen sprechen auch davon, dass es wahrscheinlich ist, dass es nie Quantencomputer geben wird. Um einen Kryptoanalytisch relevanten Quantencomputer zu bauen, der tatsächlich in der Lage ist, Chiffren wie RSA oder ECC-Varianten zu knacken, wären bis zu millionen von Qubits notwendig, ob es jemals einen Fehlerkorrigierenden Quantencomputer von dieser größe geben wird, ist noch höchst spekulativ [26].

Aus der Sicht der Theoretischen Informatik, bezeichnet man die Komplexitätsklasse der derzeitigen Quantencomputer als NISQ. NISQ beschreibt eine Komplexitätsklasse von Problemen, die mit klassischen Computern, mit Zugang zu rauschbehafteten Quanten-schaltkreisen gelöst werden können. NISQ Computer sind z.B. in der Lage, angepasste Probleme von Simon's Problem zu lösen, sie sind aber nicht in der Lage, beschleunigungen bei unstrukturierter Suche wie Grover's Algorithmus zu erreichen. Wenn man die Komplexitätsklassen vergleicht wie in Abbildung-4.3, lässt sich sehen, dass NISQ durch den Zugang zu rauschbehafteten Quantencomputern, zusätz-

lich in der Lage ist, eine geringfügige Quantenbeschleunigung für manche Probleme zu erreichen. Man definiert drei Komplexitätsklassen:

- BPP (bounded error probabilistic polynomial time)
  Die Klasse der Entscheidungsprobleme, für die es einen randomisierten Algorithmus gibt, der unter begrenzter Fehlerwahrscheinlichkeit, das Entscheidungsprobleme in polynomieller Zeit lösen kann. Ein klassischer Computer mit Zufall kann diese Probleme lösen.

- NISQ (noisy intermediate-scale quantum)
  Die Klasse der Entscheidungsprobleme, die mit klassichen Computern mit Zugriff auf einen rauschbehafteten Quantencomputern, in polynomieller Zeit, gelöst werden können.

- BQP (bounded error quantum polynomial time)
  Die Klasse der Entscheidungsprobleme die, mit einem Quantenalgorithmus, auf einem idealen Quantencomputer, unter begrenzter Fehlerwahrscheinlichkeit, in polynomieller Zeit gelöst werden können [44].

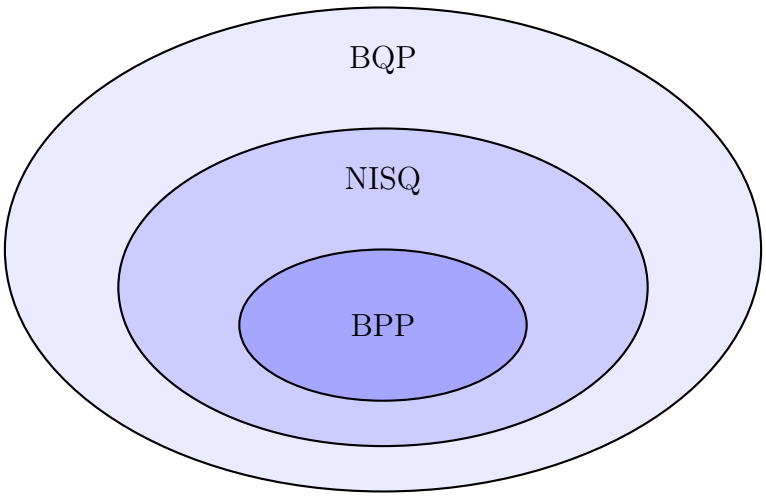


Abbildung 4.3: Verhältnis der Komplexitätsklassen BPP, NISQ und BQP. Eigene Darstellung.

Es ist nicht bekannt, welche Probleme alle in NISQ liegen, es ist allerdings bekannt, dass nicht für alle Probleme eine Quantenbeschleunigung erreicht werden kann. Manche Probleme sind effizienter auf klassischen Computern zu lösen, als auf rauschbehafteten Quantencomputern. Keines der Fragestellungen von BPP $\subsetneq$ NISQ, NISQ $\subsetneq$ BQP und BPP $\subsetneq$ BQP sind alle nicht bewiesen. Bekannt ist, dass gilt BPP $\subseteq$ NISQ, weil NISQ einen klassischen Computer vollständig beinhaltet und eine NISQ Schnittstelle hat. Ein Fehlerkorrigierender Quantencomputer muss auch in der Lage sein, einen rauschbehafteten Quantencomputer zu simulieren, deshalb, NISQ $\subseteq$ BQP. Nach der Definition von NISQ, als ein klassischer Computer mit Zugang zu rauschbehafteten Quantencomputern, haben wir: BPP $\subseteq$ NISQ $\subseteq$ BQP, siehe Abbildung-4.3.

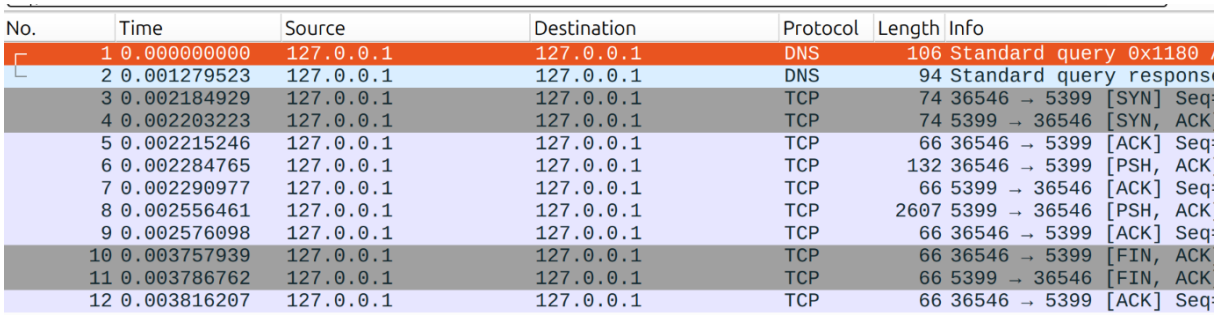
Zudem ist bewiesen, dass NISQ Computer nicht eine Beschleunigung wie Grover's Algorithmus erreichen können [35].

Erkenntnisse des PoC

Durch die Berücksichtigung von Schlüssel- und Signaturlängen, sowie Laufzeit und abweichungen, sind die PQC-Signaturalgorithmen aus der libqos bibliothek die noch in Frage kommen, bezüglich Median und Standartabweichung:

1. ML-DSA44
2. FALCON512
3. MAYO1
4. SNOVAIK

Das BSI empfiehlt als PQC-Signatur alternative für DNSSEC, ML-DSA im Merkle-Tree-Ladder Modus zu nutzen. Durch die Auswertung des PoC hat sich ergeben, dass ML-DSA nicht die performanteste Alternative ist. ML-DSA schlägt zwar SNOVAIK, manchmal in der Signaturerstellung und dessen öffentlicher Schlüssel ist 144 Byte kürzer als der von MAYO1, ansonsten hat es aber die schlechtesten Werte in jedem Fall. Zusätzlich ist durch ML-DSA44 auch ein TCP-fallback notwendig, siehe Abbildung-4.4.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	DNS	106	Standard query 0x1180
2	0.001279523	127.0.0.1	127.0.0.1	DNS	94	Standard query respons
3	0.002184929	127.0.0.1	127.0.0.1	TCP	74	36546 → 5399 [SYN] Seq
4	0.002203223	127.0.0.1	127.0.0.1	TCP	74	5399 → 36546 [SYN, ACK
5	0.002215246	127.0.0.1	127.0.0.1	TCP	66	36546 → 5399 [ACK] Seq
6	0.002284765	127.0.0.1	127.0.0.1	TCP	132	36546 → 5399 [PSH, ACK
7	0.002290977	127.0.0.1	127.0.0.1	TCP	66	5399 → 36546 [ACK] Seq
8	0.002556461	127.0.0.1	127.0.0.1	TCP	2607	5399 → 36546 [PSH, ACK
9	0.002576098	127.0.0.1	127.0.0.1	TCP	66	36546 → 5399 [ACK] Seq
10	0.003757939	127.0.0.1	127.0.0.1	TCP	66	36546 → 5399 [FIN, ACK
11	0.003786762	127.0.0.1	127.0.0.1	TCP	66	5399 → 36546 [FIN, ACK
12	0.003816207	127.0.0.1	127.0.0.1	TCP	66	36546 → 5399 [ACK] Seq

Abbildung 4.4: Screenshot, Wireshark captures eines RRSIG-Paketes mit ML-DSA44 Signatur

Für MAYO1, FALCON512 und SNOVAIK, ist immer noch UDP möglich.

Da NIST der Merkle-Tree-Ladder Modus empfiehlt, wäre es in diesem Modus nur notwendig, hin- und wieder die Ladder selbst zu signieren, ansonsten wird gezieltes hashen verwendet. Wenn nur hin- und wieder die teure PQC-Signatur eingesetzt werden muss, macht das insgesamt keinen großen Unterschied welcher Algorithmus verwendet wird. Nach diesem Prinzip könnte sogar der SHA-DSA eingesetzt werden, sollte ML-DSA kryptoanalytische Schwächen aufweisen sollte.

Dennoch zeigen MAYO1, SNOVAIK und FALCON512 bessere performance und könnte alle weiterhin mit UDP-only betrieben werden. Warum NIST diesen dennoch bevorzugt, könnte daran

liegen dass FALCON durch dessen komplexe implementierung und häufige Verwendung von Zufallszahlengeneratoren, eine hohe Wahrscheinlichkeit für implementierungsschwächen zeigt. Multivariate Verfahren wie SNOVA und MAYO haben oftmals gezeigt, trotz annahme sicher zu sein, signifikante Schwächen zu haben, wie z.B. das Verfahren „Rainbow“, welches im Laufe des NIST Prozess vollständig gebrochen wurde [32].

Quantenkryptografie

Nicht zu verwechseln mit der Post-Quanten-Kryptografie, über die bisher geschrieben wurde. Die Quantenkryptografie beschäftigt sich damit wie man die Gesetze der Quantenphysik benutzen kann um damit sichere Kryptografie zu bauen. Es ist ein besser erforschtes Feld als Quantencomputer an sich. Der BB84-Quantenschlüsselaustausch macht sich das Prinzip zunutze, dass man Photonen nicht observieren kann, ohne sie zu ändern. Was bei Qubits dazu führt dass man sie nicht verändern kann, ist hier die Sicherheit. Eine Nachricht wird über Photonen übertragen und wenn eine dritte Person diese Nachricht mitschneidet, wird sie verändert und es ist somit aufgefallen. Statt das die Sicherheit auf der Schwierigkeit eines mathematischen Problems basiert, basiert die Sicherheit hier auf den Regeln der Quantenmechanik [62, 19]. Wegen einigen Schwächen wird aber von der NSA, von der Nutzung dieser Verfahren abgeraten, allerding entwickelt NIST dieses Feld durchgehend weiter und versucht weiterhin Rauschen zu redizieren und somit die Übertragung verlässlicher zu machen [16]. Eventuell könnte sich dieses Feld in der Zukunft noch in der Asymmetrischen Kryptografie etablieren. Dadurch dass diese Verfahren auch nicht durch Angriffe Quantencomputern (wenn sicher) gebrochen werden können, haben sie auch eine Schnittmenge mit der Post-Quanten-Kryptografie. Für DNSSEC wäre allerding etwas wie „Generalized Quantum-assisted Digital Signature (GQaDS)“ interessant, da digitale Signaturen benötigt werden [69].

Zusammenfassung

Quantencomputer sind noch lange nicht auf dem Stand akut eine Gefahr darszustellen. Es ist nicht bewusst, zu was NISQ Quantencomputer in der Lage sind, es ist aber klar, dass sie keine exponetielle Beschleunigung wie Grover's Algorihmus bringen können und sie sind nicht in der Lage, in mittlerer größe, tatsächlich durch Shor's Algorithmus, wirklich Kryptografie zu brechen. Quantenalgorithmen existieren bereits, man weiß aber auch nicht ob z.B. Regev's Algorithmus in der Realität hinsichtlich Interferenz, Dekohärenz und rauschen wirklich effizienter als der von Shor ist.

Zudem basieren die Auswertungen von Ergebnissen bisher stärker aus heuristiken als tatsäch-

lich auf mathematischen Modellen (siehe Abbildung-4.1). Damit Quantencomputer praktisch relevant für Berechnungen werden, müssen verlässlicher werden oder Fortschritte in der Fehlerkorrektur erziehlen, um wirklich kryptoanalytisch relevant zu werden, müssen sie viel größer werden, im Bereich von millionen von Qubits.

Was DNSSEC betrifft, ist die Umstellung aus PQC-Signaturverfahren zwar ein notwendiger Schritt, aber wohl nicht die akuteste Bedrohung. Signaturen sind nicht so stark gefärdet wie die Vertraulichkeit von Nachrichten über symmetrische Verfahren. Eine Signatur kann wenn sie in den kritischen zeitlichen Bereich fällt, berechnet zu werden, kann einfach eine neue angefordert werden. Da mit einen Quantencomputer, der in der Lage ist, mit Shor's/Regev's- Algorithmus eine Signatur zu verfälschen, würde das immer noch Tage bis Wochen in Anspruch nehmen, mit einer gültigkeit der Signatur von wenigen Stunden, würde das notfalls vermutlich sogar noch ausreichen, als Notlösung. Andererseits für gelagerte und verschlüsselte Nachrichten, basierend auf Schlüsseln von Asymmetrischen Verfahren, sind Angriffe eine größere Gefahr.

Wenn die Umstellung auf PQC geschieht, dann werden mit hoher Wahrscheinlichkeit die ersten Probleme vermehrt implementierungsfehler, zero-days, etc. sein. Wenn die Entwicklung von Quantencomputern voranschreitet, wird ebenso die PQC ihren Platz einnehmen. Wie lange das geht, ist im optimistischen Fall, aus der Sicht des Quantencomputers, von circa 10, 15 oder Jahren auszugehen. Eine Post-Quanten Alternative muss denoch gefunden werden, auch wenn der Fortschritt erstmals stagnieren sollte. Das Risiko ist noch gering, im Ernstfall ist der worst-case aber nicht akzeptabel. Die Lösung sollte gefunden werden, bevor das Problem eingetreten ist.

Literaturverzeichnis

[1] 6. Advanced Configurations — BIND 9 9.20.29 documentation. https://bind9.readthedocs.io/en/v9.20.29/chapter6.html#dynamic-update.

[2] The Deutsch-Jozsa algorithm. https://quantum.cloud.ibm.com/learning/en/courses/fundamentals-of-quantum-algorithms/quantum-query-algorithms/quantum.cloud.ibm.com/learning/en/courses/fundame of-quantum-algorithms/quantum-query-algorithms/deutsch-jozsa-algorithm.

[3] Entwicklungsstand Quantencomputer. https://www.bsi.bund.de/DE/Themen/Unternehmen-und-Organisationen/Informationen-und-Empfehlungen/Quantentechnologien-und-Post-Quanten-Kryptografie/Entwicklungsstand-Quantencomputer/entwicklungsstand-quantencomputer.html?nn=127176.

[4] Hawk. https://hawk-sign.info/#team.

[5] Landmark IBM error correction paper on Nature cover | IBM Quantum Computing Blog. https://www.ibm.com/quantum/blog/nature-qldpc-error-correction.

[6] Lattice Signatures: Fiat-Shamir Intuition. https://www.nccgroup.com/research/building-intuition-for-lattice-based-signatures-part-2-fiat-shamir-with-aborts.

[7] Perfect binary tree. https://xlinux.nist.gov/dads/HTML/perfectBinaryTree.html.

[8] PQCRainbow. https://www.pqcrainbow.org/.

[9] Quantum computation of complex systems. https://arxiv.org/html/2211.16910v1.

[10] The query model of computation. https://quantum.cloud.ibm.com/learning/en/courses/fundamentals-of-quantum-algorithms/quantum-query-algorithms/quantum.cloud.ibm.com/learning/en/courses/fundame of-quantum-algorithms/quantum-query-algorithms/deutsch-jozsa-algorithm.

[11] Service Name and Transport Protocol Port Number Registry. https://www.iana.org/assignments/service-names-port-numbers?&page=2.

[12] SIKE – Supersingular Isogeny Key Encapsulation. http://sike.org/.

[13] Simon's algorithm. https://quantum.cloud.ibm.com/learning/en/courses/fundamentals-of-quantum-algorithms/quantum-query-algorithms/quantum.cloud.ibm.com/learning/en/courses/fundamenta of-quantum-algorithms/quantum-query-algorithms/simon-algorithm.

[14] SNOVA — Post-Quantum Digital Signature Scheme (NIST Round 3). https://snova.pqclab.org/.

[15] What CIOs Need to Know About Quantum Computing. https://www.gartner.com/en/articles/quantum-computing-readiness.

[16] What Is Quantum Cryptography? *NIST*, April 2025.

[17] Kubitti. *Wikipedia*, September 2026.

[18] Moonshotai/Kimi-K3 · Hugging Face. https://huggingface.co/moonshotai/Kimi-K3, July 2026.

[19] Olli Ahonen. Entanglement-Enhanced Quantum Key Distribution, March 2009.

[20] Gorjan Alagic, Maxime Bros, Pierre Ciadoux, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status report on the fourth round of the NIST post-quantum cryptography standardization process. Technical Report NIST IR 8545, National Institute of Standards and Technology (U.S.), Gaithersburg, MD, March 2025.

[21] Anthropic. Discovering cryptographic weaknesses with Claude. https://www.anthropic.com/research/discovering-cryptographic-weaknesses, July 2026.

[22] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose. DNS Security Introduction and Requirements. Request for Comments RFC 4033, Internet Engineering Task Force, March 2005.

[23] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose. Protocol Modifications for the DNS Security Extensions. Request for Comments RFC 4035, Internet Engineering Task Force, March 2005.

[24] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose. Resource Records for the DNS Security Extensions. Request for Comments RFC 4034, Internet Engineering Task Force, March 2005.

[25] asudbring. Übersicht über DNSSEC: Azure Public DNS. https://learn.microsoft.com/de-de/azure/dns/dnssec.

[26] Jean-Philippe Aumasson. *Serious Cryptography: A Practical Introduction to Modern Encryption*. No Starch Press, San Francisco, 2018.

[27] Marco Baldi, Alessandro Barenghi, Politecnico di Milano, Michele Battagliola, Sebastian Bitzer, Marco Gianvecchio, Politecnico di Milano, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi, Politecnico di Milano, Federico Pintore, Paolo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia Wachter-Zeh, and Violetta Weger. Algorithm Specifications and Supporting Documentation. 2025.

[28] Boaz Barak. *An Intensive Introduction to Cryptography: Lattice Based Cryptography*.

[29] Christian Baun. *Computernetze kompakt: eine praxisorientierte Einführung für Studium und Beruf*. IT kompakt. Springer Vieweg, Wiesbaden, 7. auflage edition, 2025.

[30] Ryad Benadjila, Thibauld Feneuil, and Matthieu Rivain. MQ on my Mind: Post-Quantum Signatures from the Non-Structured Multivariate Quadratic Problem, 2023.

[31] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS$^+$ Signature Framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2129–2146, London United Kingdom, November 2019. ACM.

[32] Ward Beullens. Breaking Rainbow Takes a Weekend on a Laptop, 2022.

[33] Big Think. Michio Kaku: Quantum computing is the next revolution, August 2023.

[34] Jordana Cepelewicz. 'Post-Quantum' Cryptography Scheme Is Cracked on a Laptop, August 2022.

[35] Sitan Chen, Jordan Cotler, Hsin-Yuan Huang, and Jerry Li. The complexity of NISQ. *Nature Communications*, 14(1):6001, September 2023.

[36] Information Technology Laboratory Computer Security Division. Round 3 Submissions - Post-Quantum Cryptography | CSRC | CSRC. https://csrc.nist.gov/Projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions, January 2017.

[37] David A. Cooper, Daniel C. Apon, Quynh H. Dang, Michael S. Davidson, Morris J. Dworkin, and Carl A. Miller. Recommendation for Stateful Hash-Based Signature Schemes. Technical report, National Institute of Standards and Technology, October 2020.

[38] Quantum Daily. Qilimanjaro Inaugurates an Analog Quantum Computer at Barcelona Supercomputing Center. https://quantumnews.in/article/quantum-computing/qilimanjaro-inaugurates-analog-quantum-computer-barcelona-supercomputing-center, May 2026.

[39] Jintai Ding and Dieter Schmidt. Rainbow, a New Multivariable Polynomial Signature Scheme. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi, Gerhard Weikum, John Ioannidis, Angelos Keromytis, and Moti Yung, editors, *Applied Cryptography and Network Security*, volume 3531, pages 164–175. Springer Berlin Heidelberg, Berlin, Heidelberg, 2005.

[40] Andrew Fregly, Joseph Harvey, Burton S. Kaliski, and Swapneel Sheth. Merkle Tree Ladder Mode: Reducing the Size Impact of NIST PQC Signature Algorithms in Practice. In Mike Rosulek, editor, *Topics in Cryptology – CT-RSA 2023*, volume 13871, pages 415–441. Springer International Publishing, Cham, 2023.

[41] Jürgen Fuß and Anna Vymazal. *Kryptographie*. Springer Berlin Heidelberg, Berlin, Heidelberg, 2025.

[42] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing - STOC '96*, pages 212–219, Philadelphia, Pennsylvania, United States, 1996. ACM Press.

[43] Joe Harvey, Burt Kaliski, Anew Fregly, Swapneel Sheth, and D. McVicker. Merkle Tree Ladder (MTL) Mode Signatures. Internet Draft draft-harvey-cfrg-mtl-mode-09, Internet Engineering Task Force, March 2026.

[44] Matthias Homeister. *Quantum Computing verstehen: Grundlagen - Anwendungen - Perspektiven*. Computational Intelligence. Springer Vieweg, Wiesbaden, 4. aufl. 2015 edition, 2015.

[45] S. Huque, C. Elmerot, and O. Gudmundsson. Compact Denial of Existence in DNSSEC. Request for Comments RFC 9824, Internet Engineering Task Force, September 2025.

[46] IBM. Deutsch's algorithm. https://quantum.cloud.ibm.com/learning/en/courses/fundamentals-

of-quantum-algorithms/quantum-query-algorithms/quantum.cloud.ibm.com/learning/en/courses/fundame
of-quantum-algorithms/quantum-query-algorithms/deutsch-algorithm.

[47] Marcel Luthi and Gabriele Roger. Algorithmen und Datenstrukturen.

[48] Macchiavello Artur Ekert, Hosgood Timothy, Kay Alastair, and Chiara. *2.10 The Bloch Sphere | Introduction to Quantum Information Science*.

[49] P. Mockapetris. Domain names - concepts and facilities. Request for Comments RFC 1034, Internet Engineering Task Force, November 1987.

[50] P. Mockapetris. Domain names - implementation and specification. Request for Comments RFC 1035, Internet Engineering Task Force, November 1987.

[51] Giuseppe Di Molfetta. Quantum Computing : A New Frontier for Science and Society, July 2026.

[52] Dustin Moody, Gorjan Alagic, Maxime Bros, Pierre Ciadoux, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl Miller, Rene Peralta, Ray Perlner, Angela Robinson, Hamilton Silberg, Daniel Smith-Tone, and Noah Waller. Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process. NIST Interagency/Internal Report (NISTIR) 8610, National Institute of Standards and Technology, Gaithersburg, MD, May 2026.

[53] Duc Tri Nguyen and Kris Gaj. Fast Falcon Signature Generation and Verification Using ARMv8 NEON Instructions. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology - AFRICACRYPT 2023*, volume 14064, pages 417–441. Springer Nature Switzerland, Cham, 2023.

[54] Christof Paar and Jan Pelzl. *Kryptografie verständlich: Ein Lehrbuch für Studierende und Anwender*. eXamen.press. Springer Berlin Heidelberg, Berlin, Heidelberg, 1. aufl. 2016 edition, 2016.

[55] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. Falcon. https://falcon-sign.info/.

[56] Gesche Pospiech. *Quantencomputer and Co: Grundideen und Zentrale Begriffe der Quanteninformation Verständlich Erklärt*. Essentials Ser. Springer Fachmedien Wiesbaden GmbH, Wiesbaden, 2021.

[57] Oded Regev. An Efficient Quantum Factoring Algorithm, January 2024.

[58] Scott Rose, Cricket Liu, and Ross Gibson. Secure Domain Name System (DNS) Deployment Guide:. Technical Report 800-81r3, National Institute of Standards and Technology, Gaithersburg, MD, 2026.

[59] Markku-Juhani Saarinen. *Post-Quantum Cryptography: 15th International Workshop, PQCrypto 2024, Oxford, UK, June 12-14, 2024, Proceedings, Part II*. Number v.14772 in Lecture Notes in Computer Science Series. Springer, Cham, 1st ed edition, 2024.

[60] Klaus Schmeh. Die Fiat-Shamir-Transformation. In Klaus Schmeh, editor, *Post-Quanten-Kryptografie: Eine anschauliche Einführung*, pages 71–76. Springer Fachmedien, Wiesbaden, 2026.

[61] Klaus Schmeh. Gitter-basierte Algorithmen. In Klaus Schmeh, editor, *Post-Quanten-Kryptografie: Eine anschauliche Einführung*, pages 77–97. Springer Fachmedien, Wiesbaden, 2026.

[62] Klaus Schmeh. Quantencomputer. In Klaus Schmeh, editor, *Post-Quanten-Kryptografie: Eine anschauliche Einführung*, pages 27–46. Springer Fachmedien, Wiesbaden, 2026.

[63] Peter Schwabe. Dilithium. https://pq-crystals.org/dilithium/index.shtml.

[64] Peter W. Shor. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997.

[65] Josh Schneider Smalley, Ian. Was ist Quantencomputing? | IBM. https://www.ibm.com/de-de/think/topics/quantum-computing, September 2025.

[66] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marc Manzano, and Ramesh Karri. *Hardware Architectures for Post-Quantum Digital Signature Schemes*. Springer International Publishing, Cham, 1st ed. 2021 edition, 2021.

[67] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marcos Manzano, and Ramesh Karri. FALCON. In Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marc Manzano, and Ramesh Karri, editors, *Hardware Architectures for Post-Quantum Digital Signature Schemes*, pages 31–41. Springer International Publishing, Cham, 2021.

[68] Zygimantas Straznickas and Stephen A Weis. HAWK-nKey Recovery Reduces to SVP in Dimensionn/2 +.

[69] Alberto Tarable, Rudi Paolo Paganelli, Elisabetta Storelli, Alberto Gatto, and Marco Ferrari. Generalized Quantum-assisted Digital Signature, June 2024.

[70] MAYO team. MAYO. https://pqmayo.org/.

[71] P. Vixie. Extension Mechanisms for DNS (EDNS0). Request for Comments RFC 2671, Internet Engineering Task Force, August 1999.